论文编号：_____

广州商学院

本科毕业论文（设计）

题　　目：基于流程引擎和协同过滤算法的房屋租赁系统设计与实现

姓　　名：洪文豪

学　　号：202212390611

学　　院：现代信息产业学院

专业班级：软件工程（产教融合创新班专升本）软工华为 2206

指导教师：杨茹

2026 年 6 月

本科毕业论文（设计）原创性声明

本人郑重声明：所呈交的毕业论文（设计），是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品或成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。本声明的法律结果由本人承担。

作者签名：洪文豪　　日期：2026 年 6 月 1 日

本科毕业论文（设计）版权使用授权书

本论文（设计）作者完全了解学校有关保留、使用毕业论文（设计）的规定，同意学校保留并向国家有关部门或机构送交论文（设计）的复印件和电子版，允许论文（设计）被查阅和借阅。本人授权广州商学院可以将本论文（设计）的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本论文（设计）。

本论文（设计）属于

1．保密□，在_____年解密后适用本授权书。

2．不保密 ☑。

（请在以上相应方框内打“√”）

作者签名：洪文豪　　日期：　2026 年 6 月 1 日

指导老师签名：　　日期：　2026 年 6 月 1 日

摘要

鉴于城镇化加速以及青年租住群体扩大，租赁市场对信息透明、流程规范以及风险防控提出了更高要求。传统的线下招租以及人工撮合导致了房东身份模糊、费用披露缺失、合同约束不足，使得租客决策成本上升。

本文设计并且实现基于 Spring Boot 的房屋租赁系统，选用 B/S 架构，后端运用 Spring Boot 3.x 以及 MyBatis-Plus，前端使用 Vue 3 搭配 Element Plus，存储选用 MySQL 8.0，借助 Redis 来缓存热点数据。系统面向租客、房东以及管理员，涵盖了用户服务、房源管理、租赁业务、合同审批、消息通知以及后台管理这六大子系统。

核心创新包括：提出“三类房东＋五项费用”双透明模型，强制标注一手房东、转租房东以及中介属性并且披露水电燃气物业网络计价方式；构建融合协同过滤推荐以及内容相似度的混合推荐，引入用户行为得以实现房源匹配；集成 Activiti 7 流程引擎来驱动审核以及签约流转，结合 RabbitMQ 进行异步通知；开发规则库驱动的轻量合同风控，去识别押金模糊、维修责任缺失等风险条款。测试表明系统在并发场景下运行稳定，风控覆盖常见风险，契合信息透明以及流程合规的目标。

关键词：房屋租赁系统；Spring Boot；协同过滤推荐；流程引擎；合同风控；信息透明

Abstract

Given the acceleration of urbanization and the growing population of young renters, the rental market now demands greater transparency, standardized processes, and enhanced risk management. Traditional offline rental advertising and manual brokerage have led to unclear landlord identities, a lack of fee disclosure, and insufficient contractual safeguards, thereby increasing the decision-making costs for tenants.

This paper designs and implements a housing rental system based on Spring Boot. It adopts a B/S architecture, utilizing Spring Boot 3.x and MyBatis-Plus for the backend, Vue 3 paired with Element Plus for the frontend, and MySQL 8.0 for storage, with Redis used to cache frequently accessed data. The system serves tenants, landlords, and administrators, encompassing six major subsystems: user services, property management, leasing operations, contract approval, message notifications, and backend administration.

Core innovations include: proposing a "three-category landlords + five-fee" dual-transparency model, which mandates the identification of primary landlords, subletting landlords, and real estate agents, and requires the disclosure of pricing methods for utilities (water, electricity, gas) and property management fees; Building a hybrid recommendation system that integrates collaborative filtering and content similarity, incorporating user behavior to enable property matching; integrating the Activiti 7 workflow engine to drive approval and contract signing workflows, combined with RabbitMQ for asynchronous notifications; developing a rule-based, lightweight contract risk control system to identify risky clauses such as ambiguous deposit terms and missing repair liability provisions. Testing indicates that the system operates stably under concurrent scenarios, with risk control covering common risks, aligning with the goals of information transparency and process compliance.

Key words: Housing Rental System; Spring Boot; Collaborative Filtering Recommendation; Process Engine; Contract Risk Control; Information Transparency

目录

摘要.....I
Abstract.....II
目录.....III
1 绪论.....1
1.1 课题研究的背景及意义.....1
1.1.1 课题背景.....1
1.1.2 项目的意义.....1
1.2 与本课题相关的国内外现状.....2
1.3 论文组织.....2
2 关键理论与技术介绍.....4
2.1 Spring Boot 框架.....4
2.2 Vue 3 与 Element Plus.....4
2.3 MyBatis-Plus.....4
2.4 MySQL 数据库.....4
2.5 Redis 缓存.....5
2.6 RabbitMQ 消息队列.....5
2.7 Activiti 流程引擎.....5
2.8 协同过滤推荐算法.....5
2.9 Java 技术.....5
3 系统分析与设计.....7
3.1 可行性分析.....7
3.1.1 技术可行性.....7
3.1.2 经济可行性.....7
3.1.3 操作可行性.....7
3.2 系统功能需求分析.....8
3.2.1 租客功能需求.....8
3.2.2 房东功能需求.....8
3.2.3 管理员功能需求.....8
3.3 系统非功能需求分析.....8
3.4 用例分析.....9
3.5 系统业务流程分析.....11
3.6 数据流分析.....12
3.7 系统设计概述.....13
3.8 数据库概念结构设计.....13
3.8.1 数据库总体 E-R 图.....13
3.8.2 各实体 E-R 图.....14
3.9 数据库逻辑结构设计.....19
3.10本章小结.....26
4 系统实现.....28
4.1 用户认证模块实现.....28

4.1.1 登录界面 28
4.1.2 注册界面 28
4.1.3 个人中心界面 29
4.1.4 认证核心代码 30
4.2 房源管理模块实现 32
4.2.1 房源搜索列表界面 32
4.2.2 Redis 缓存热门房源核心代码 33
4.3 首页与智能推荐模块实现 35
4.3.1 协同过滤推荐算法 36
4.4 合同管理与风险识别模块实现 37
4.4.1 合同风险识别规则 37
4.4.2 风控服务核心代码 38
4.4.3 合同生成与签署接口 40
4.5 消息通知模块实现 42
4.6 后台管理模块实现 43
4.7 Activiti 流程引擎集成 44
5 系统测试 48
5.1 测试目的 48
5.2 测试资源 48
5.3 测试方法 48
5.4 测试用例及执行 48
5.4.1 用户认证模块测试用例 48
5.4.2 房源管理模块测试用例 49
5.4.3 租赁业务模块测试用例 49
5.4.4 后台管理模块测试用例 50
5.4.5 安全性测试用例 51
5.5 测试报告 51
6 总结与展望 53
6.1 研究工作总结 53
6.2 主要功能与创新点 53
6.3 不足之处 53
6.4 后续展望 54
参考文献 55
致谢 57

1 绪论

1.1 课题研究的背景及意义

1.1.1 课题背景

随着我国城镇化建设的深入开展，大量的劳动力在城市中进行集聚，推动了住房租赁需求水涨船高[1]。依据住建部 2024 年发布的统计报告显示，全国租房群体规模已经突破了 2.6 亿，在这当中，“90 后”以及“00 后”的占比超过了六成。但是，在这个庞大的市场背景下，传统的租赁模式显露出了诸多的弊病。在很多城市当中，不管是像一线还是三四线，房东依旧依靠在街头去张贴“出租”字条、或者运用朋友圈开展信息的发布工作来招徕租客，租房流程过于复杂，参与的租赁双方需要投入大量的时间成本和更多的额外支出，这种方式信息传播效率低下，交易流程不够透明，导致了双方的沟通成本居高不下[2]。

同时，现有的互联网租赁平台尽管在房源的聚合以及展示这方面开展了一定的工作，可是在关键信息的披露方面还是存在着明显的不足。拿贝壳、自如这些头部平台来说，房源页面往往没有对“一手房东直租”、“二手房东转租”以及“中介代管”这三种截然不同的出租身份去进行醒目的区分，这导致租客在浏览房源的时候难以迅速地判断对方身份，所产生的信息差会直接影响到租住决策。此外，诸如水费、电费以及燃气费这类日常开支的计价方式，现实中在大多数平台上都缺乏统一的标注，租客往往在入住之后才察觉实际费用与预期不符，租客遭遇押金纠纷的情况屡见不鲜，房东常以“门禁费”、“卫生费”、“自然损耗”等名义扣减押金，这类情况时有发生，也致使各类纠纷频繁出现[3]。根据中消协发布的《2023 年住房租赁服务投诉分析报告》显示，涉及虚假房源、费用欺诈以及合同陷阱的投诉量同比上升了 37%，其中青年群体的投诉占比更是高达 68%。

就技术层面而言，鉴于近年来 Spring Boot、Vue 这类主流开发框架日趋成熟，同时协同过滤推荐算法以及 Activiti 流程引擎也在各类互联网平台中得到了广泛验证，这为去构建一套兼顾信息透明以及智能服务的房屋租赁系统奠定了坚实的技术基础。 因此，本课题拟运用 Spring Boot 以及 Vue 技术栈，来对一个面向国内市场的房屋租赁系统进行设计与实现。所以，本课题拟借助 Spring Boot 以及 Vue 技术栈，来设计并且实现一个面向国内市场的房屋租赁平台，力求得以在房东身份标识、费用结构公示、智能推荐匹配、流程自动审批以及合同风险预警这些环节实现系统性的改进。

1.1.2 项目的意义

鉴于租客立场，其急需一个能提供真实、透明房源信息的平台。依靠在前端强制展示房东类别即一手、二手房东或持牌中介，以及水、电、燃气、物业、网络五项费用标准，可消除信息盲区，助租客在签约前建立准确成本预期。房东同样能运用系统开展房源发布、审批申请以及收益管理工作，来提高管理效率。立足行业视角，系统引入基于用户画像以及房源标签的混合推荐算法，把协同过滤和内容相似度计算结合，能提高匹配精度并减少

无效沟通。同时，集成 Activiti 引擎来驱动合同签署等节点的自动化流转，还凭借 RabbitMQ 得以实现预约确认等异步消息推送，有助于规范流程、降低风险。此外，轻量级合同风控模块可基于预设规则库自动检测像“押金退还条件模糊”这种高风险条款并预警，来保护双方权益。

从学术层面，本课题把 Spring Boot、Vue、MySQL、Redis、RabbitMQ、Activiti 等组件整合到租赁场景当中，探索了“可信信息+智能服务”驱动的建设思路，对推动行业数字化转型以及信息化升级有参考价值。

1.2 与本课题相关的国内外现状

国内领域，这几年的研究与实践工作主要是基于“规范治理+平台优化”这两条路径来开展的。政策研究比较着重强调住房租赁市场的法治化以及高质量发展，在于扩充住房租赁需求供给，提供法治化路径和明确的政策预期，指出需要在主体责任、交易规则以及信息披露等方面形成更为明确的约束[4]。在系统实现层面，鉴于运用 Spring Boot 与 MySQL 开展的租赁平台建设、智能匹配算法落地等工作不断增多；个性化推荐综述也给协同过滤与内容融合技术提供了方法支撑[5]。同时，工作流规则库与消息驱动机制、RabbitMQ 异步处理等技术也被证明可以提高流程规范性与系统并发能力[6]。不过目前的成果主要还是把单模块改进当作重点，在房东身份显性化、费用标准统一展示、推荐与风控协同联动等方面，还缺少面向完整业务闭环的一体化方案。

国外研究则更加重视租赁市场运行机制以及对数据模型的应用。相关文献从租金刚性、消费波动、租金管制以及租客应对行为等角度出发，演变规律进行了梳理，揭示了租赁关系当中的结构性问题，这给平台治理提供了经济学与社会学等理论基础，应重视配套政策举措的完善和租金管制模式的选择[7]。在算法层面，基于用户浏览行为、检索历史和兴趣偏好，采用协同过滤、内容分析以及实时个性化推荐在多类在线服务场景中已形成了较成熟的方法体系，这对租房场景实现精准匹配拥有借鉴价值[8]。

总体来看，国外经验在模型方法与治理框架上拥有参考意义，但如果把其直接移植到国内，依然会受到监管环境、数据基础以及实施成本的约束。怎样结合本土条件来实现“身份透明、费用透明、流程可控、风险前置”的系统化落地，依然是本课题的核心切入点。

1.3 论文组织

全文共划分成了六个章节，具体的安排情况如下所示：

第一章为绪论，主要阐述了课题的研究背景、项目意义以及国内外相关的研究现状，并且说明了论文的整体结构。

第二章为关键理论与技术介绍，针对系统开发过程所涉及的 Spring Boot、Vue、MyBatis-Plus、MySQL、Redis、RabbitMQ、Activiti、Java 等关键技术进行了概述。

第三章为系统分析与设计，借助用户需求和功能需求这两个维度展开，明确系统业务范围和功能边界，囊括总体架构设计、功能模块划分、数据库设计以及核心业务流程设计。

第四章为系统实现，展示了各个功能模块具体的实现过程。

第五章为系统测试相关报告的展示。

第六章为总结与展望，回顾了全文工作，对研究成果进行了总结，分析了不足之处并且提出了未来的改进方向。

2 关键理论与技术介绍

2.1 Spring Boot 框架

Spring Boot 是 Pivotal 团队基于 Spring 生态所推出的一个轻量级开源快速开发框架，它的设计初衷是为了简化 Spring 项目从搭建、配置、部署的整个过程，帮助开发者快速构建项目[9]。跟传统的 Spring 项目来比，Spring Boot 最显著的特性就是“约定优于配置”——开发人员不必去编写海量的 XML 配置文件，框架会在内部凭借自动装配机制把常用组件的默认配置都预置好，只有在存在特殊需求的时候才去进行人工覆盖。本系统选用了 Spring Boot 3.1.5 版本，在 Java 17 环境下运行，借助它内嵌的 Tomcat 容器来实现一键启动，同时也运用 starter 依赖管理来快速引入 Spring Security、Spring Data Redis 以及 Spring AMQP 等功能模块，这让后端开发人员可以把主要精力都集中在业务逻辑的编写工作上，而不是去重复搭建底层基础设施。

2.2 Vue 3 与 Element Plus

Vue 作为一套用来构建用户界面的渐进式 JavaScript 框架，3.x 版本引入了 Composition API，让开发者得以运用函数组合的方式去组织组件逻辑，比起 Options API，它在复杂组件中拥有更好的可读性与复用性[10]。本系统前端基于 Vue 3.4 以及 Vite 5 来构建，Vite 借助浏览器原生 ES Module 来实现按需编译，开发时的热更新速度比起 Webpack 快了一个数量级。状态管理选用 Pinia 来替代 Vuex，它的类型推断更友好、API 更精简，适宜中等规模的单页应用。UI 层选用 Element Plus 2.4 组件库，它拥有表格、表单等 200 多个开箱即用的组件，配合 Axios 去完成前后端 HTTP 通信，前端借助 Vite 代理配置把 API 请求转发给 Spring Boot 后端，从而解决了跨域问题。另外，系统还引入了 ECharts 5.4 来实现房源数据、价格走势这类运营指标的可视化呈现。

2.3 MyBatis-Plus

MyBatis-Plus 是基于 MyBatis 来封装的一个增强工具包，其主旨是“只做增强不做改变”[11]。它内置拥有通用的 CRUD 接口以及条件构造器、分页插件，开发人员进行单表操作时几乎不需去手写 SQL 语句，减少了许多重复代码，同时也保留了 MyBatis 原生 XML 映射的能力来应对复杂查询工作[12]。本系统选用了 MyBatis-Plus 3.5.5 版本，凭借其 LambdaQueryWrapper 来实现类型安全方式的查询条件拼接，把字符串硬编码可能带来的拼写风险给避免了。逻辑删除方面，框架借助注解@TableLogic 会自动把 deleted=0 条件加到查询中，让数据在业务层面上“消失”而在物理层面上依然可以追溯，从而契合合同以及订单等场景下的审计要求。

2.4 MySQL 数据库

MySQL 属于目前运用极为广泛的开源关系型数据库，它的 InnoDB 引擎能够支持事务、行级锁以及外键约束，以此在并发场景中保障数据的一致性[13]。本系统选用了 MySQL 8.0 版本，并把字符集设为 utf8mb4 来兼容中文以及表情符号。数据库中总共开展了 8 张核心

业务表的设计工作，囊括了用户表、房源表、房源图片表、订单表、合同表、消息表、用户行为表以及评价表，各表之间借助外键关联构建了清晰的实体关系网，且在高频查询字段上建立索引来提升检索效率。合同表内的风险条目字段运用 JSON 列类型去存储，充分借助了 MySQL 8.0 对 JSON 数据的原生查询能力。

2.5 Redis 缓存

Redis 是一种鉴于内存的键值数据库，读写延迟处于微秒级，适宜把它当作热点数据缓存层来使用[14]。系统借助 Spring Data Redis 来整合 Lettuce 客户端以连接 Redis，并对连接池进行配置也就是设定最大活跃连接为 8 个，以此来实现 TCP 链路的复用。在实际业务当中，会把热门房源列表这类频繁访问的数据写入 Redis 同时设定 30 分钟过期时间，利用 Redis 特性，记录用户每日首次登录，有效减轻了 MySQL 在高并发下的读取压力。序列化策略选用了 Jackson JSON 格式，方便在排查问题时直接利用命令行工具去查看缓存内容。

2.6 RabbitMQ 消息队列

RabbitMQ 作为运用 AMQP 协议的消息中间件，利于在分布式系统中去进行异步通信以及流量削峰，保证数据和事务一致性[15]。系统对 Topic 类型交换机 house.exchange 进行了定义，下设预约、合同、订单三条消息队列，分别与 appointment.、contract.、order.*路由键进行绑定。当租客去提交看房预约或合同状态有了变更，后端只需把消息投递到交换机就能马上返回响应，消费者在后台异步地去完成站内信推送以及邮件通知，既对用户等待时间进行了缩短，又实现了通知逻辑与核心业务的解耦，减低了模块间相互的影响。

2.7 Activiti 流程引擎

Activiti 作为一款轻量级的 BPMN 2.0 流程引擎，能把业务审批流程采用可视化流程图形式去定义并驱动执行，降低流程开发门槛[16]。本系统借助 Activiti 对关键流程进行管理：合同签署流程，租客以及房东按顺序完成电子签章后合同才生效。流程定义采用 XML 文件描述，运行时由引擎自动去创建任务实例、分配处理人并且记录流转日志，使得审批状态的追踪以及回溯透明可查，规避了人工协调带来的遗漏以及推诿。

2.8 协同过滤推荐算法

协同过滤属于推荐系统领域当中应用最为广泛的算法思路之一，它核心的假设也就是“兴趣相近的用户倾向于对同类物品做出相似的评价”，通过用户的评分或用户的浏览历史行为等进行推荐[17]。本系统选用的是基于用户的协同过滤策略：首先从用户行为表中去提取每位用户的浏览、收藏、下单以及其他交互记录，并且赋予不同行为差异化的权重分值，比如浏览 1 分、收藏 3 分、下单 5 分；随后去计算当前用户和其他用户在房源交互集合上的重叠度，来找出兴趣最相似的若干邻居用户；最后把邻居用户关注过但是当前用户尚未接触的房源按照热度排序后来进行推送。要是候选房源不足，系统会自动回退到热门房源来补充推荐列表，以此确保推荐结果始终可用。

2.9 Java 技术

Java 语言在当前的后端应用开发领域中被运用得极为广泛，它能够实现“一次编写、

处处运行”也就是 Write Once, Run Anywhere 这种跨平台特性，其主要原因囊括了 JVM 即 Java 虚拟机机制：源代码会经过编译器统一去生成字节码，然后再交给各个平台上所安装的 JVM 去负责解释并执行，这样也就消除了不同操作系统之间所存在的运行差异[18]。除了拥有跨平台能力之外，Java 还内置了垃圾回收机制，会由 JVM 去自动开展堆内存的分配与释放管理工作，这在一定程度上降低了像内存泄漏这类问题发生的概率。

Java 语言是基于面向对象程序设计来开展的，封装、继承以及多态属于它的三个基本特征。

封装就是把类的属性以及操作这些属性的方法给绑定在同一个类当中，同时借助访问控制修饰符，比如 private 或者 protected，来对外部隐藏内部的实现细节，仅仅对外暴露那些必要的公共接口。这样做的益处就在于，外部代码没法去随意修改对象的内部状态，类的职责边界也因此变得更加清晰，有助于实现各模块之间耦合程度的降低，便于后期的维护工作。

继承机制允许子类在父类已有的属性和主要方法基础上去进行扩展，子类可以把父类的功能当作基础来直接复用，也可以基于实际的需要去对特定方法进行重写。要是有多个类存在着相似的逻辑，就可以把共同的部分提炼到父类当中，避免在每个子类里重复地去编写相同的代码，从而提升开发的效率并且减少潜在的不一致问题。

多态指的是同一个父类类型的引用变量在运行的时候可以指向不同的子类对象，程序会根据该引用实际指向的对象类型去动态调用对应的方法实现。这个机制使得调用方不需要针对每一种具体类型去分别编写处理逻辑，凭借统一的接口就可以去操作多种对象，提高了代码的灵活性以及可扩展性。

在这个系统的后端设计当中，封装、继承以及多态均得到了具体的应用。各个功能模块的 Service 实现类通过实现对应的接口来与 Controller 层进行解耦，Entity 类对外屏蔽了数据库字段的直接访问，而不同业务场景底下的用户角色，像租客、房东、管理员，在权限校验逻辑方面也借助了继承以及多态来减少重复的判断，从整体上来讲使得代码结构比较清晰，便于后续功能的维护以及扩展。

3 系统分析与设计

系统分析在软件开发过程当中属于承上启下的关键环节，它核心的任务就是把用户的业务诉求转变成明确的技术需求描述，从而为后续的系统设计以及编码实现打下基础。本章主要围绕房屋租赁系统真实的业务场景，从可行性论证、功能与非功能需求梳理、业务流程建模以及数据流向分析这几个方面开展系统性的分析工作，力求全面地覆盖系统的核心业务逻辑。

3.1 可行性分析

在正式去开展开发工作以前，很有必要基于技术条件、经济成本以及实际操作这三个维度，来对本系统的可行性开展审慎的评估工作。开展可行性分析的目的并不在于去证明系统一定能成功，而是要尽量把可能的风险点给识别出来，这样便于在设计阶段提前去进行规避。

3.1.1 技术可行性

该系统后端采用了 Spring Boot 3.x 框架，它历经多年社区迭代，在国内外众多生产环境当中已获充分验证。持久层运用 MyBatis-Plus，能凭借较少代码量去实现对 MySQL 8.0 数据库的增删改查操作。前端基于 Vue 3 配合 Element Plus 组件库来进行构建，在单页面应用开发效率以及用户交互体验方面表现稳定。

中间件层面，依靠 Redis 来实现热点房源数据的缓存加速，借助 RabbitMQ 去解耦业务通知的异步推送，同时运用 Activiti 7 流程引擎来驱动合同签署等审批流转[19]。上述技术组件均为开源项目，社区文档丰富、学习成本可控，且彼此间的集成方案已有成熟参考案例。因此从技术角度看，本系统的开发工作是具备可行性的。

3.1.2 经济可行性

此项目作为毕业设计作品，主要是在个人计算机上去开展开发工作的，运用的 Spring Boot、Vue、MySQL、Redis、RabbitMQ 以及 Activiti 等核心组件均为开源软件，并不会产生许可费用。测试阶段可以依靠本地环境或免费的云服务试用额度去完成。系统上线后要是需部署到云服务器，鉴于主流云厂商的定价，一台 2 核 4G 实例年费不超过千元，针对小规模运营而言经济负担较轻。支付功能方面，开发阶段运用支付宝沙箱环境来进行模拟，不需要额外资金投入。综合来看，系统在经济层面具备可行性。

3.1.3 操作可行性

鉴于最终用户视角，系统面向的租客以及房东群体，通常拥有基本的运用智能手机或电脑的能力。前端界面运用了响应式设计，页面布局简洁直观，核心功能比如房源搜索、预约下单以及合同签约，操作步骤均控制在 3 到 5 步以内即可完成。对于管理员角色，后台管理界面主要把表格以及图表作为核心，借助可视化面板便能掌握平台的运营数据。同时，系统还内置了站内信以及消息通知机制，能在关键业务节点进行自动推送提醒，降低

人工开展跟进工作的成本。因此，基于操作层面来看，各类用户都能比较顺畅地去使用本系统。

3.2 系统功能需求分析

开展功能需求分析的主要目的，在于把业务场景当中各类角色的操作诉求拆解成为具体的功能点。该系统涵盖了租客、房东以及管理员这三种角色，它们各自承担着不一样的业务职责，下面将来分别进行阐述。

3.2.1 租客功能需求

租客作为系统的主要使用群体，核心需求囊括了这么几个方面：第一，开展账户管理，涵盖手机号或邮箱注册、登录、密码找回以及实名认证；第二，进行房源浏览与检索，能借助区域、价格区间、户型、面积、房东类型也就是一手二手房东和中介等多条件来筛选，并在详情页中把水电燃气物业网络费的计价方式与参考均值明确展示出来；第三，租赁交易，这包括发起去预约看房、在线签订电子合同、缴纳押金以及租金；第四，个性化推荐，系统基于用户浏览、收藏和下单等行为记录，结合协同过滤与内容相似度算法，来自动推送契合偏好的房源；第五，信息反馈，租客能在订单完成后针对房源和房东开展评分评价，并去查看自身的信用分；第六，信用分机制，当租客当天在同一个房源累计取消订单 10 次，租客信用分减 10 分，当租客信用分等于 0 时，不能预约看房。每日首次登录信用分加 1 分。

3.2.2 房东功能需求

房东充当房源的供给端，其功能需求主要囊括以下方面：第一，进行用户身份认证，第二，进行房源管理，这涵盖录入像地址、面积、户型、装修档次这类基本信息，实施房源图片上传、五项费用明细填写以及房东类型标签设置，并针对房源去执行上架与下架的操作；第三，租赁审批，需对租客发起的预约请求开展审核，进而决定同意还是拒绝；第四，合同及收益的管理，能够在线去签署电子合同、对历史订单与收益统计进行查看；第五，信用分机制，当用户评价小于 3 颗星时，房东信用分减 10 分，评价等于 3 颗星时，房东信用分加 3 分，评价等于 4 颗星时，房东信用分加 4 分，评价等于 5 颗星时，房东信用分加 5 分，当房东信用分等于 0 时，不能发布房源。每日首次登录信用分加 1 分。

3.2.3 管理员功能需求

管理员负责开展平台的日常运营以及秩序维护工作，它的功能需求囊括：用户管理，也就是查看用户列表、对违规用户进行封禁或者解封；订单以及合同监管，去查看全平台的订单与合同状态；数据统计以及可视化，主要是借助 ECharts 图表展示房源数量、成交量、活跃用户数、热门区域分布、信用分分布等运营指标；以及系统消息的管理。

3.3 系统非功能需求分析

除了前面提到的功能需求外，系统还得满足一些非功能性的约束。性能方面，房源搜

索接口需要进行优化，得把常规并发状态下的响应时间控制在 500 毫秒以内，而针对热门的查询，则凭借 Redis 这一缓存技术来实现将延迟降低至百毫秒级别的目标。在安全方面，认证环节选用的是 JWT 机制来进行处理，密码这类敏感信息运用了 BCrypt 算法开展加密存储工作，对于身份证等关键字段，在传输以及存储的环节当中都做了脱敏处理，接口层也配置了拦截机制来防止越权访问[20]。至于可靠性，消息推送功能依靠 RabbitMQ 队列来进行中转，即便下游的服务暂时出现了不可用的情况，消息也不会发生丢失。扩展性上，系统按照模块开展了独立 Service 层的划分工作，模块之间借助 RESTful API 来进行交互，以后如果想要拆分微服务，也能以比较低的成本来实现扩展。在易用性方面，前端设计契合简洁的规范，选用了紫蓝渐变的色调，从而确保操作的路径能够清晰明了。

3.4 用例分析

租客属于本系统中数量最多、使用频次最高的角色。它的核心业务链条囊括：对房源进行浏览、筛选、查看详情、进行收藏或发起租赁意向、预约看房以及签署电子合同。在这一过程当中，系统还会基于用户行为数据，像浏览、收藏、下单记录，借助协同过滤算法来生成个性化推荐，协助租客更迅速地找到契合的房源。个人中心囊括了个人信息修改、预约管理、我的收藏、合同管理以及信用分查看等子功能，把实名认证当作可选拓展用例，能提升用户的信用分，如图 3-1 所示。

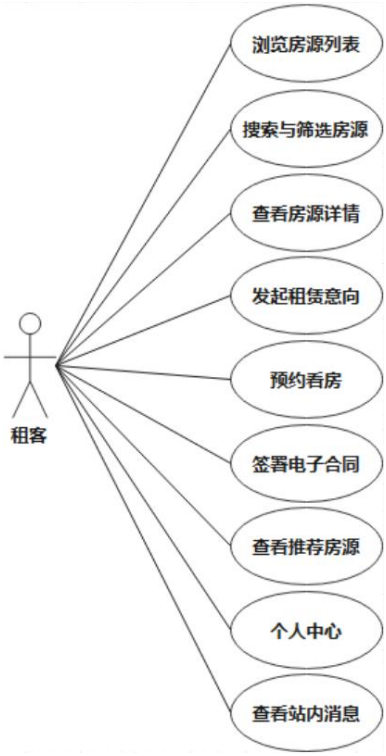

图 3-1 租客用例图

房东作为房源的供给方，它核心业务主要是围绕“房源管理”以及“订单审批”这两条主线来开展的。在房源管理工作当中，房东需要去填写房源标题、描述、户型参数，还有包含水、电、燃气、物业、网络在内的五项生活费用明细，同时标明像一手房东、二手

房东以及中介这样的房东类型，在订单审批工作上，房东可以对租客所发起的看房预约开展审批工作，通过之后系统会自动把合同草稿生成出来，房东和租客在确认条款无误后便能完成签署，如图 3-2 所示。

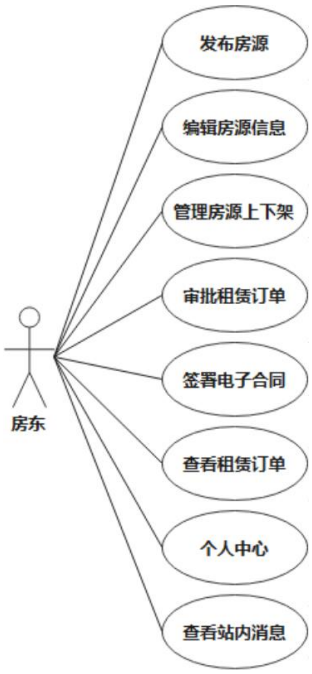

图 3-2 房东用例图

管理员作为系统的维护者以及监管者，具备对全平台用户以及业务数据开展管理的权限。在用户管理方面，管理员能够去查看用户列表、搜索指定用户，并且对违规账号进行封禁或者解封操作。此外，管理员还可以查看系统当中的全部订单以及合同记录，并且借助数据统计面板来获取用户数量、房源数量、订单量以及合同量等运营指标，从而为平台的精细化运营提供数据支撑，如图 3-3 所示。

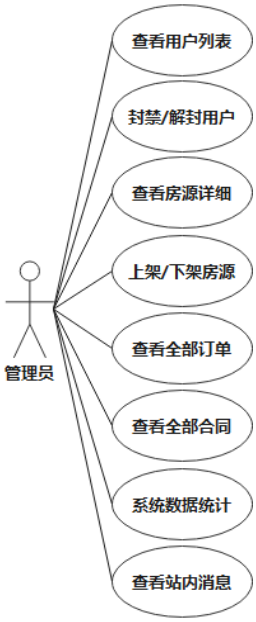

图 3-3 管理员用例图

3.5 系统业务流程分析

为了直观地来展示系统核心业务逻辑，下面基于租客以及房东两条主线，去梳理租赁业务的完整流程，并借助流程图来呈现。

租客端流程是：用户完成注册登录并且通过实名认证后，进入房源检索页面，能凭借区域、价格、户型以及房东类型等条件来筛选。点击某房源进详情页，页面上方会展示房东属性，即一手房东、二手房东或中介，以及五项生活费用的明细。如果租客对该房源有意向，就可以发起预约看房请求。这个请求提交后，系统会借助 RabbitMQ 把通知推送给房东。房东审批通过后，租客由房东带去看房，租客看房后明确租赁房屋，房东就可以生成电子合同，双方进入电子合同签署环节，合同文本由系统自动生成并且经过规则引擎去进行风险条款检测。租客签署后，房东才能进行签署，双方签署完毕后，租客缴纳押金以及首月租金，订单状态流转为已完成，租客能够在此基础上去对房源进行评价，如图 3-4 所示。

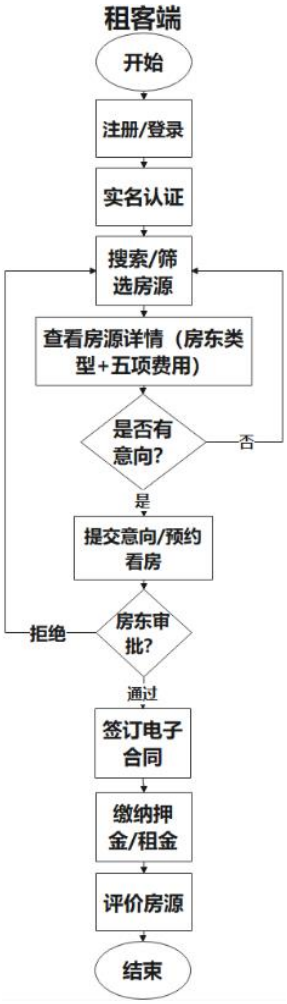

图 3-4 租客端系统业务流程图

房东端的流程：房东完成注册并认证身份之后，需在发布房源页面去录入房屋基础信息、上传图片并标注类型，同时录入五项费用的计价标准。收到租客请求之后，房东在个

人中心开展审批，同意后系统会自动拉起合同签署流程。签署完成之后，房东能在收益管理模块当中查看订单详情与收入统计，如图 3-5 所示。

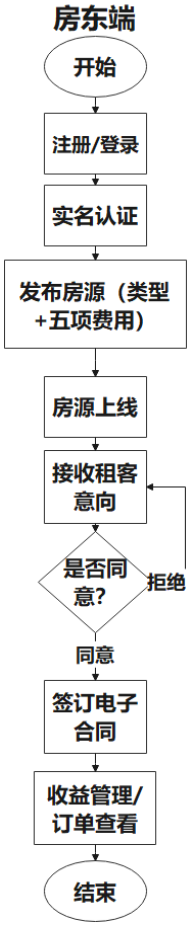

图 3-5 房东端系统业务流程图

3.6 数据流分析

数据流图鉴于数据在系统当中的流转角度开展，描绘了各个处理模块跟外部实体、数据存储之间进行信息交换的关系。该系统的顶层数据流图涉及三个外部实体，也就是租客、房东以及管理员，还涵盖六个核心处理过程以及五个数据存储。

租客作为外部实体，会把注册信息、房源搜索条件、租赁意向和合同签署请求提交给系统，同时从系统获取房源列表、推荐结果、订单状态及消息通知。房东会向系统提交身份认证资料、房源发布信息以及审批决策，并从系统接收租客请求通知和收益报表。管理员会对系统开展用户管理、房源管理和合同审计等操作，来获取平台运营统计数据。

在系统内部，P1 用户管理模块负责去处理用户注册、登录以及认证逻辑，它的数据读写是在 D1 用户数据存储；P2 房源管理模块负责对房源进行发布、编辑和检索，数据存取是在 D2 房源数据存储；P3 订单管理模块去处理意向提交、预约看房及订单流转，P4 合同管理模块负责合同生成、风险检测和签署流程，这两者的数据共同落入到 D4 订单与合同存储；P5 推荐引擎从 D3 行为数据存储当中读取用户浏览、收藏等行为记录，计算相似度后把推荐结果返回给前端；P6 消息通知模块借助 RabbitMQ 来接收各业务模块产生的通知

事件，写入到 D5 消息存储后推送至目标用户，如图 3-6 所示。

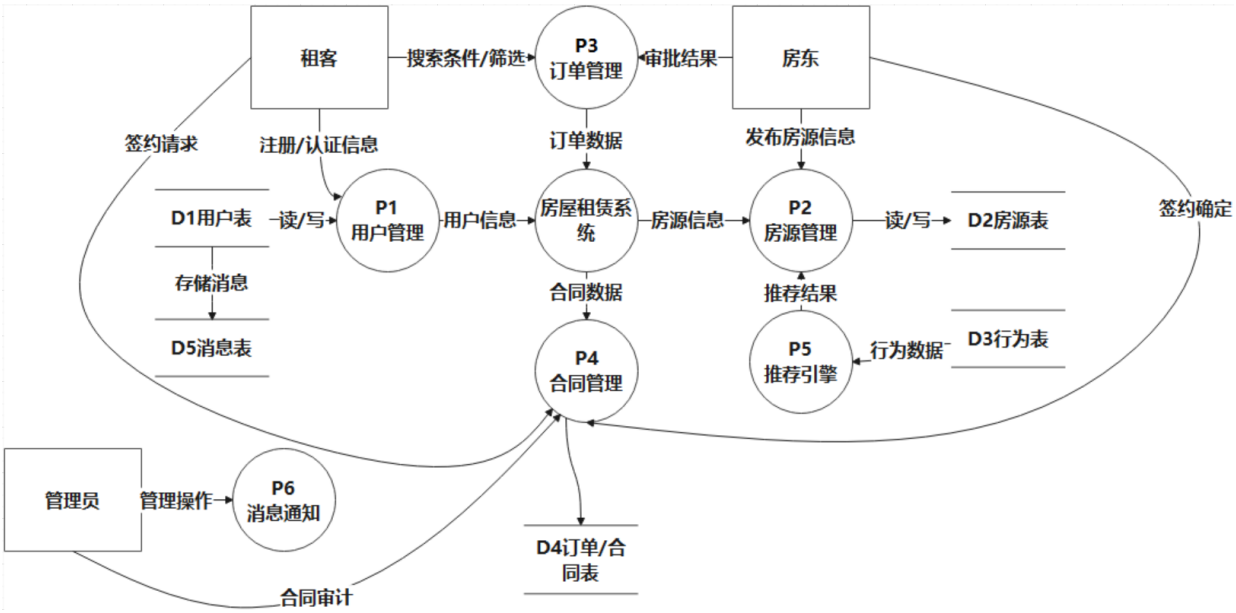

图 3-6 系统数据流图（顶层）

3.7 系统设计概述

系统设计环节充当了对接需求分析与编码实现的桥梁，它直接关联到平台运行的稳定度以及后续功能拓展的灵活性。本章重点阐述房屋租赁系统在数据库层面的概念结构与逻辑结构。概念结构设计选用实体-联系模型也就是 E-R Model 来刻画业务对象间的语义关联，逻辑结构设计则把概念模型映射成 MySQL 关系表，详细列出每张表的字段名、数据类型、约束条件及中英文含义。这两个阶段相互印证，共同去保障数据存储方案在完整性、一致性和可维护性方面契合实际业务的要求。

本系统的数据库被命名为 house_leasing，选用 UTF-8MB4 字符编码来兼容中文及表情符号。全库共规划了 8 张核心业务表：用户表 users、房源表 houses、房源图片表 house_images、订单表 orders、合同表 contracts、站内消息表 messages、用户行为记录表 user_behaviors 以及房源评价表 reviews。各表之间凭借外键约束来建立关联，并且在关键列上建立了索引以此来提升查询性能。

3.8 数据库概念结构设计

概念结构设计的核心工作在于去理清业务领域当中的各个实体以及相互关系，并借助 E-R 图来进行可视化的呈现。在这个系统里，“用户”是位于数据模型的中心位置的，和其余各实体都有着直接或者是间接的联系。

3.8.1 数据库总体 E-R 图

如图 3-7 所示，展示出了本系统的数据库总体 E-R 图。图中的矩形代表了实体，连线上标注的“1”和“N”分别代表着一端以及多端。从整体来看，用户即 users 实体与房源即 houses 之间构成了“一对多”的关系，也就是说一个房东可以发布多套房源；用户和订

单即 orders 之间也是“一对多”，一位租客能产生若干订单；订单与合同即 contracts 之间则是一对一的对应关系；房源和房源图片即 house_images 之间属于“一对多”，一套房源可以上传多张展示图片。此外，用户的浏览、收藏等行为记录在用户行为表 user_behaviors 当中，租住后的评价存储在评价表 reviews 里，站内通知则汇集在消息表 messages 之中。上述实体以及联系共同组成了租赁业务数据的骨架。

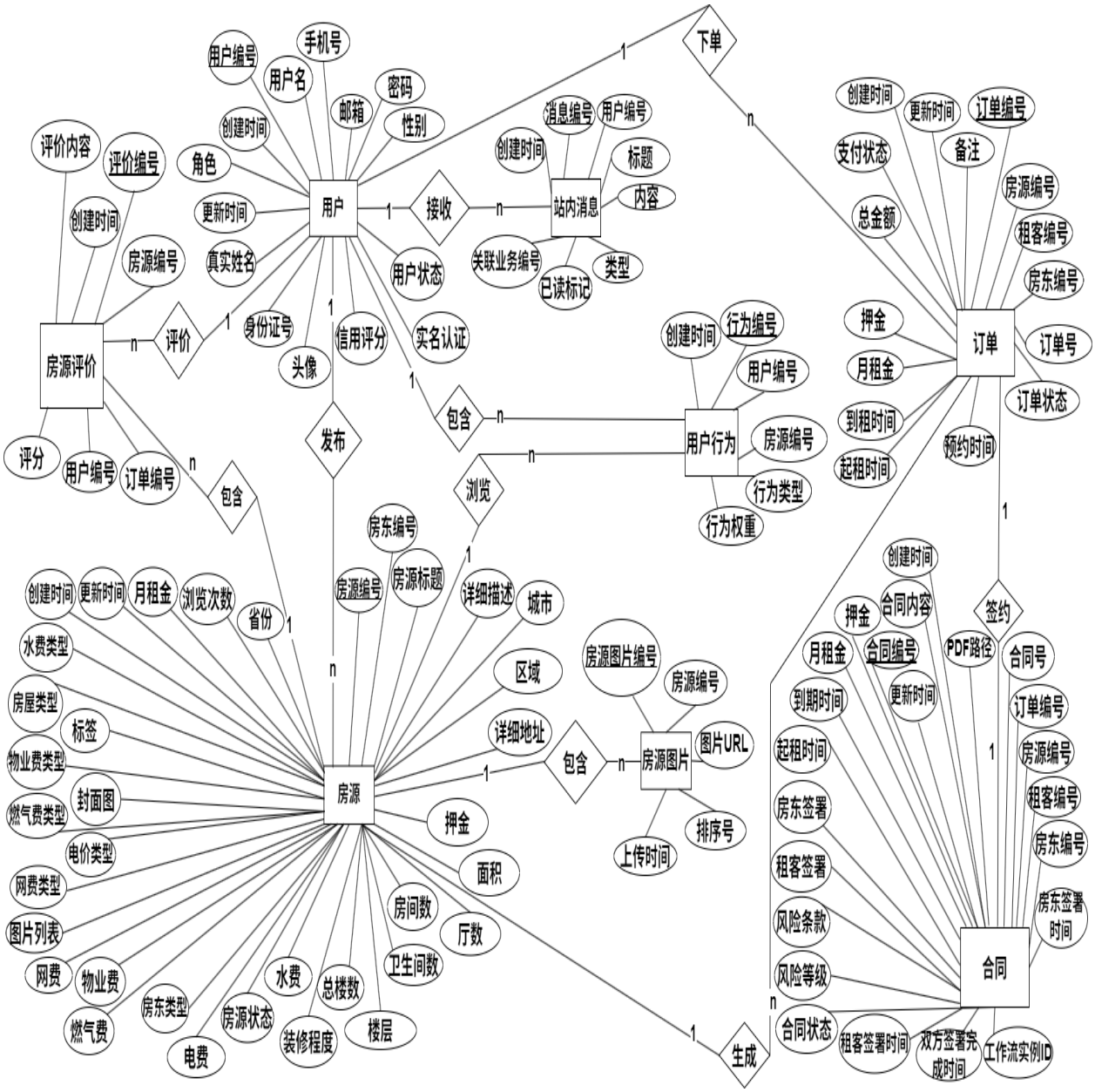

图 3-7 房屋租赁系统数据库总体 E-R 图

3.8.2 各实体 E-R 图

用户实体拥有用户名、手机号、邮箱、密码、角色、真实姓名、身份证号、头像以及信用分等属性。role 字段借助枚举类型来区分租客即 TENANT、房东即 LANDLORD 以及管理员即 ADMIN 三种角色，credit_score 字段用来记录用户信用分，初始值设为 100 分且上限 200 分，用来开展后续的信用评估以及风控工作，如图 3-8 所示。

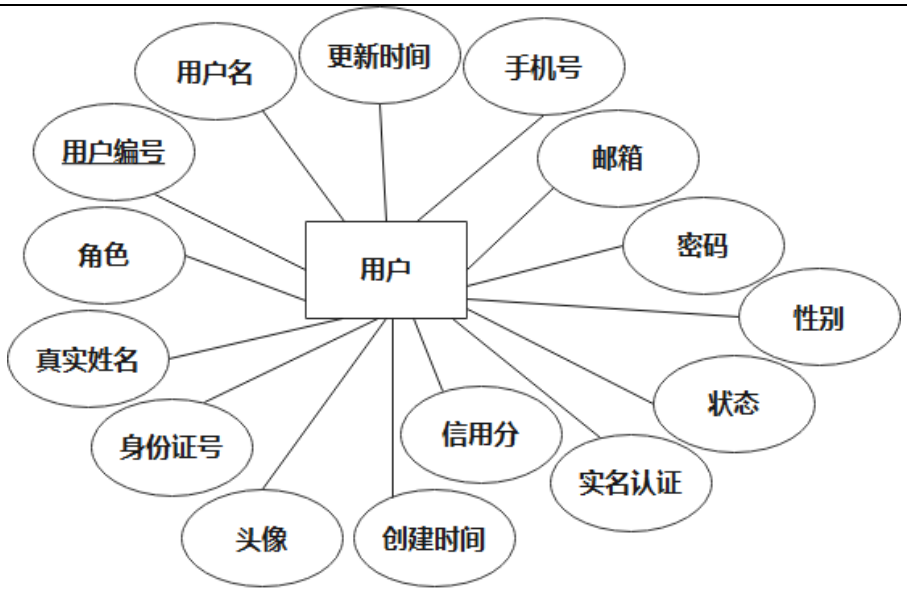

图 3-8 用户实体 E-R 图（users）

房源实体充当了体系当中属性最为丰富的实体，它涵盖了房源基本信息、户型参数以及费用明细、状态管理等多个方面的维度。值得关注的是，owner_type 字段凭借枚举值 OWNER、SUBLEASE 以及 AGENT，分别去对一手房东、二手房东（即转租方）以及持牌中介进行标识，而这也是本系统“三类房东”透明模型在数据层面的支撑。针对水、电、燃气、物业、网络这五项生活费用，系统分别为其设置了单价字段以及计费方式字段，且计费方式被划分为按表计量也就是 METERED、固定金额也就是 FIXED 以及费用包含即 INCLUDED 这三种，借此来满足不同收费场景下的需要，如图 3-9 所示。

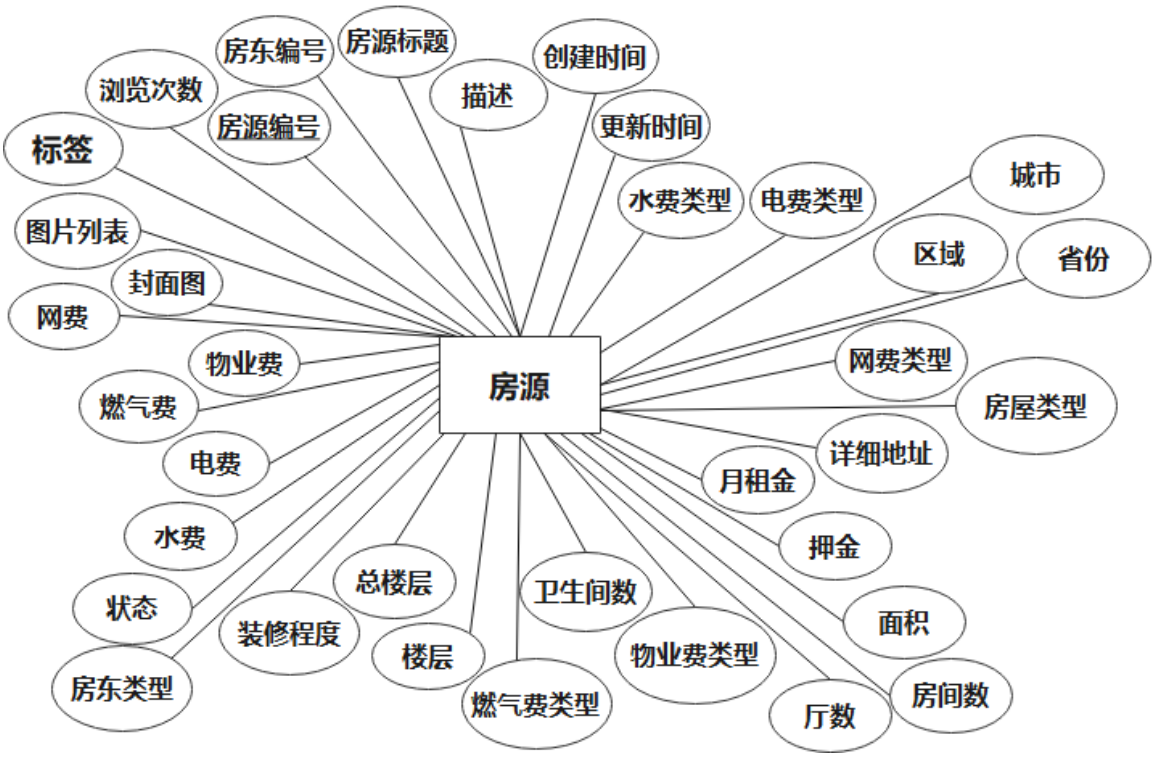

图 3-9 房源实体 E-R 图（houses）

房源图片实体结构比较简洁，主要去记录图片和房源的归属关系还有展示顺序。借助 sort 字段去控制图片在前端页面的显示次序。该表和房源表之间借助外键约束构建级联删除关系，若某套房源被移除，它关联的所有图片记录就会自动清除，如图 3-10 所示。

图 3-10 房源图片实体 E-R 图（house_images）

订单实体会对租客以及房东间的交易信息进行记录。status 字段会对订单从“待处理”到“已完成”的全生命周期进行追踪。同时，payment_status 字段独立对支付状态进行记录，支持未支付、已支付以及已退款三种情形，如图 3-11 所示。

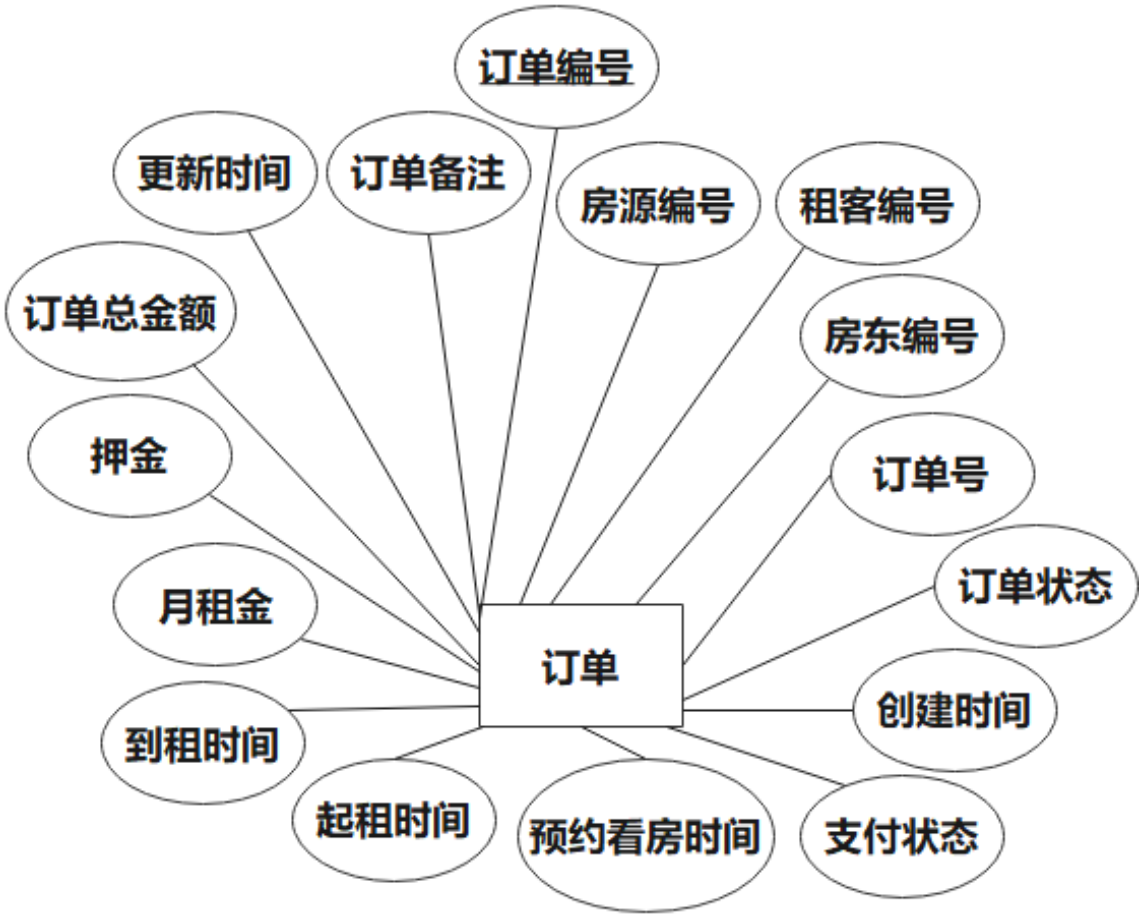

图 3-11 订单实体 E-R 图（orders）

合同实体主要用来对电子租赁合同的全过程进行管理。合同状态从草稿也就是 DRAFT 开始，经历待签署、单方签署直至双方签署完成即 FULLY_SIGNED。

风控字段 risk_level 与 risk_items 分别用来记录系统自动扫描条款后的风险等级以及风险条目，以此来为双方提供预警。

pdf_path 字段用来存储 PDF 归档路径，workflow_instance_id 字段则用于关联审批流程实例，如图 3-12 所示。

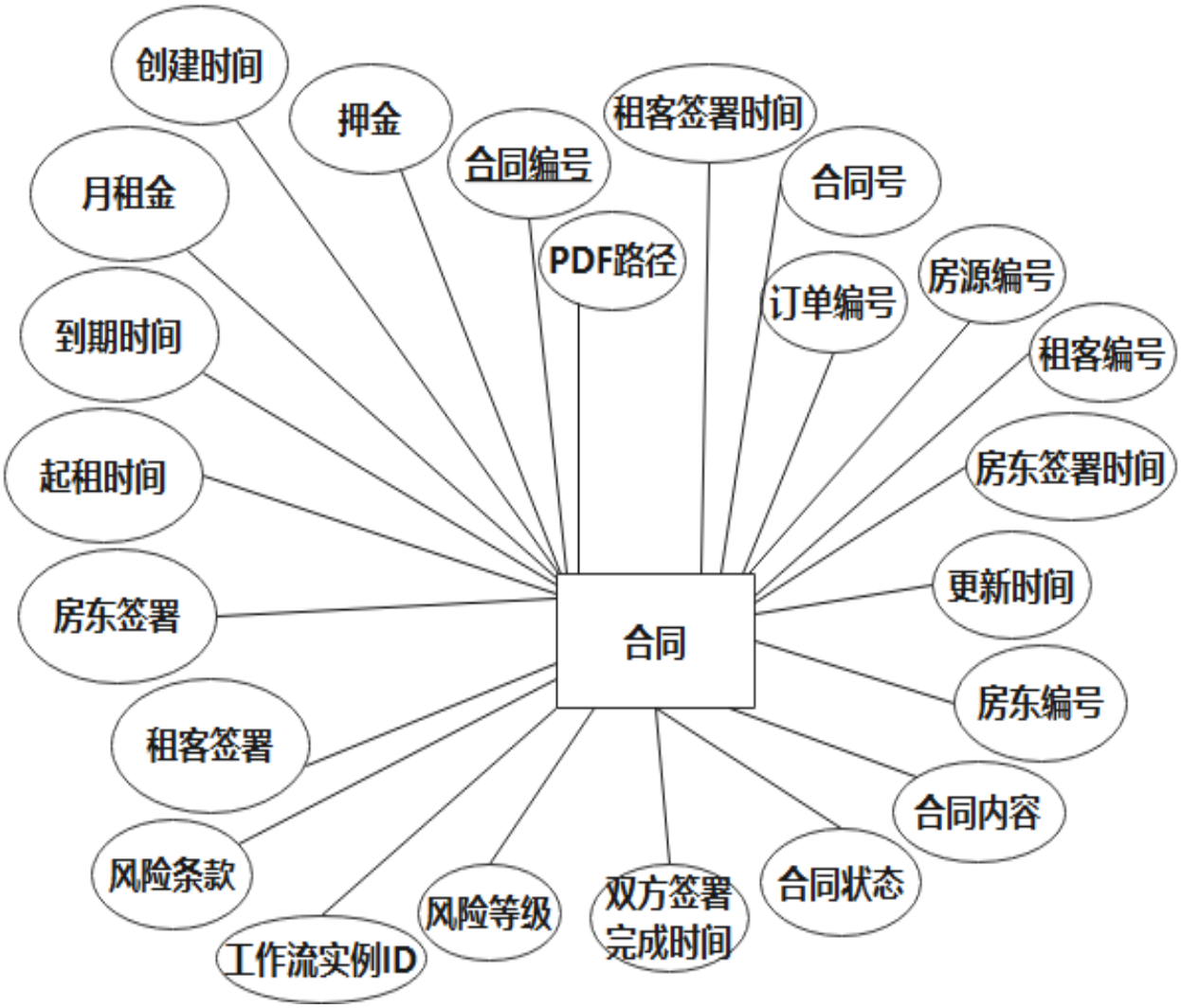

图 3-12 合同实体 E-R 图（contracts）

站内消息实体主要负责开展系统通知以及用户间消息传递的工作。借助 type 字段能把消息分作五种类别。

即系统通知 SYSTEM、订单通知 ORDER、合同通知 CONTRACT、预约通知 APPOINTMENT 和评价通知 REVIEW。

related_id 字段用于关联触发消息的业务记录，便于用户在点击消息后直接跳转至对应的业务详情页面，如图 3-13 所示。

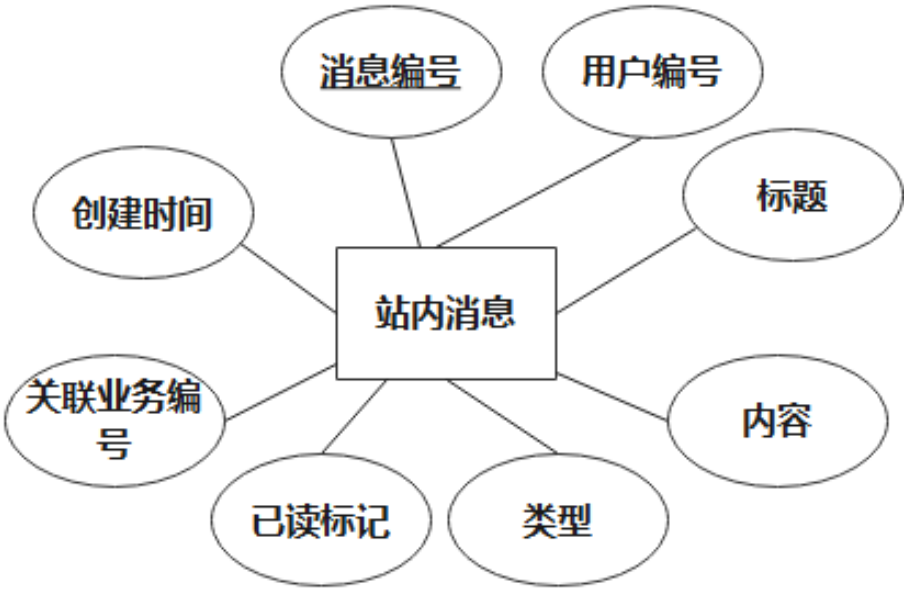

图 3-13 站内消息实体 E-R 图（messages）

用户行为实体属于推荐算法的数据基础。系统会对用户针对房源的四种行为进行记录，即浏览 VIEW、收藏 COLLECT、下单 ORDER，并且把不同的权重分值分配给各行为，其中浏览权重是 1.0，收藏为 3.0，下单是 5.0。协同过滤算法鉴于表中的行为数据来构建用户-房源评分矩阵，进而去给租客提供个性化的房源推荐如图 3-14 所示。

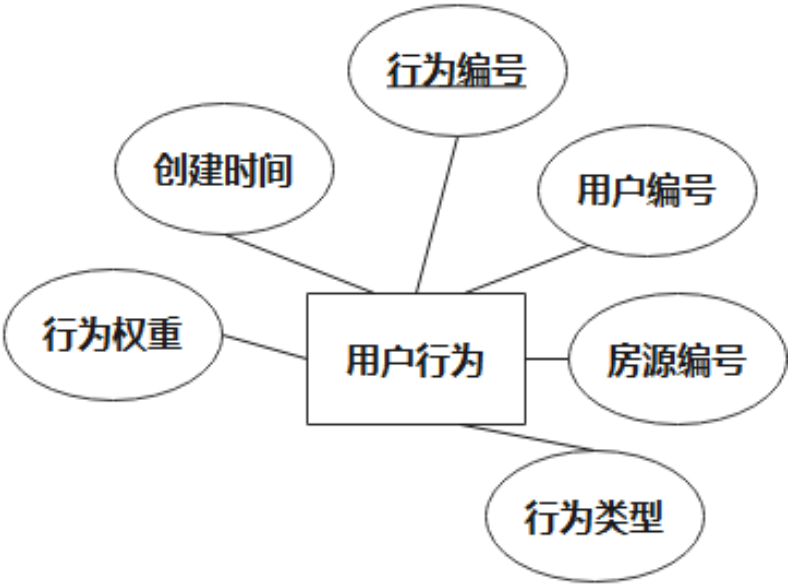

图 3-14 用户行为实体 E-R 图（user_behaviors）

评价实体记录了租客租赁后对房源的评分以及文字点评。rating 字段运用 1 至 5 星整数评分，content 字段则存储评价正文。

此表将房源与用户进行关联，既能基于房源维度汇总评价来辅助租客决策，又能基于用户维度去查看个人评价历史，如图 3-15 所示。

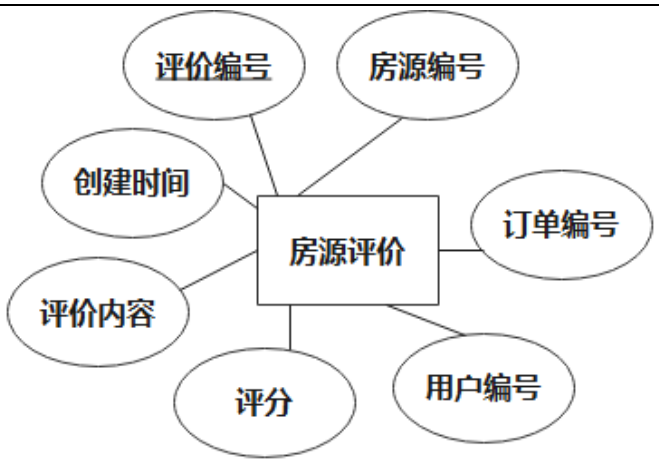

图 3-15 房源评价实体 E-R 图（reviews）

3.9 数据库逻辑结构设计

用户表属于系统当中较为核心的基础表，用来存储所有注册用户的账户信息以及身份数据。该消息生产者模块主要负责开展向 RabbitMQ 发送各类业务通知消息的工作。一旦遇到 RabbitMQ 不可用的情况，它会自动执行降级操作，也就是改为直接写入数据库，以此来确保消息的可靠性。RabbitTemplate 乃是运用在发送消息到队列这一过程中的 RabbitMQ 模板。MessageService 被当作消息服务来使用，在 RabbitMQ 服务不可用的时候，会直接去调用这个服务来进行消息的保存。发送预约确认通知消息这一功能，是为了实现通知租客其预约申请已经提交，需等待房东确认。如果 RabbitMQ 处于可用状态，就会把消息发送到队列中去进行异步处理；要是不可用，则会直接把消息保存到数据库里。发送合同状态变更通知消息这一功能，旨在通知用户关于合同的最新状态。基于 RabbitMQ 可用的前提，会把消息发往队列开展异步处理；若是不可用，系统则会把消息暂存到本地也就是数据库中，以等待后续的处理工作。发送订单状态变更通知消息的功能，是用来通知用户订单的最新状态信息的。如果 RabbitMQ 可用，就将消息发送至队列；要是不可用，则直接把消息保存到数据库当中。降级处理的具体方式是直接去调用消息服务，以此来实现将消息保存到数据库的操作。构建 RabbitMQ 消息体 Map 的这个方法，最终会返回一个包含了消息数据的 Map 对象，如表格 3-1 所示。

表格 3-1 用户表结构（users）

序号	列名	数据类型	长度	约束	允许空	说明
1	id	BIGINT		主键	否	用户 ID
2	username	VARCHAR	50	唯一	否	用户名
3	password	VARCHAR	255		否	密码
4	email	VARCHAR	100	唯一	否	邮箱
5	phone	VARCHAR	20	唯一	否	手机号
6	role	ENUM		默认值	是	用户角色，（默认值为 TENANT）

续表格 3-1 用户表结构（users）

序号	列名	数据类型	长度	约束	允许空	说明
7	real_name	VARCHAR	50		是	真实姓名
8	id_card	VARCHAR	18		是	身份证号码
9	avatar	VARCHAR	500		是	头像 URL
10	credit_score	INT		默认值	是	信用分（0-200），（默认值为 100）
11	is_real_name_auth	TINYINT		默认值	是	实名认证，（默认值为 0）
12	gender	TINYINT		默认值	是	性别，（默认值为 0）
13	status	ENUM		默认值	是	账号状态，（默认值为 ACTIVE）
14	create_time	DATETIME			是	创建时间
15	update_time	DATETIME			是	更新时间

房源表被视作系统中字段最为庞大、业务内涵最为深厚的数据表，它承载了房屋租赁交易这一核心的所有关键信息。该表主要从多个维度来描绘房源的全貌：在基本信息维度上，包含了标题、描述、所在城市、具体区域以及详细地址；在户型参数维度上，囊括了面积大小、室厅卫数量、楼层高度以及装修程度，也就是从 ROUGH 至 LUXURY 分出的五档；费用明细维度则设定了水费、电费、燃气费、物业费以及网络费这五项生活开支，且为每项费用都配备了对应的计费方式字段 fee_type，其取值涵盖 METERED 即按表计费、FIXED 即固定费用，以及 INCLUDED 即包含在租金当中，前端页面会基于这些数据动态地进行费用明细卡片的渲染，以此协助租客在签约前充分掌握实际居住成本。owner_type 字段是本系统“三类房东透明模型”的关键支撑，凭借 OWNER 即一手房东、SUBLEASE 即二手房东或转租方，以及 AGENT 即持牌中介这三个枚举值，在信息层面向租客披露房源实际出租方的真实身份。status 字段负责对房源的全生命周期状态进行管理，状态流转囊括了从 ONLINE 已上架到 OFFLINE 已下架。此外，images 字段选用 JSON 格式来存储多张图片的 URL 地址，tags 字段则运用逗号分隔的标签形式以方便后续的搜索匹配。如表格 3-2 所示。

表格 3-2 房源表结构（houses）

序号	列名	数据类型	长度	约束	允许空	说明
1	id	BIGINT		主键	否	房源 ID
2	title	VARCHAR	200		否	房源标题
3	description	TEXT			是	详细描述
4	city	VARCHAR	50		否	所在城市
5	district	VARCHAR	50		否	所在区域

续表格 3-2 房源表结构（houses）

序号	列名	数据类型	长度	约束	允许空	说明
6	address	VARCHAR	255		否	详细地址
7	price	DECIMAL	10,2		否	月租金（元）
8	deposit	DECIMAL	10,2		是	押金（元）
9	area	DECIMAL	6,2		是	面积（平方米）
10	rooms	INT		默认值	是	房间数量，（默认值为 1）
11	halls	INT		默认值	是	客厅数量，（默认值为 1）
12	bathrooms	INT		默认值	是	卫生间数量，（默认值为 1）
13	floor	INT			是	所在楼层
14	total_floor	INT			是	总楼层数
15	decoration	ENUM		默认值	是	装修程度，（默认值为 SIMPLE）
16	house_type	ENUM		默认值	是	房屋类型，（默认值为 APARTMENT）
17	owner_type	ENUM			否	房东类型（一手/二手/中介）
18	status	ENUM		默认值	是	房源状态，（默认值为 ONLINE）
19	water_fee	DECIMAL	6,2		是	水费单价
20	water_fee_type	ENUM		默认值	是	水费计费方式，（默认值为 METERED）
21	electric_fee	DECIMAL	6,2		是	电费单价
22	electric_fee_type	ENUM		默认值	默认值	电费计费方式（默认值为 METERED）
23	gas_fee	DECIMAL	6,2		是	燃气费单价
24	gas_fee_type	ENUM		默认值	是	燃气费计费方式，（默认值为 FIXED）
25	property_fee	DECIMAL	8,2		是	物业费（月）
26	property_fee_type	ENUM		默认值	是	物业费计费方式，（默认值为 FIXED）
27	internet_fee	DECIMAL	6,2		是	网费（月）
28	internet_fee_type	ENUM		默认值	是	网费计费方式，（默认值为 FIXED）

续表格 3-2 房源表结构（houses）

序号	列名	数据类型	长度	约束	允许空	说明
29	view_count	INT		默认值	是	浏览次数，（默认值为 0）
30	owner_id	BIGINT		外键	否	房东用户 ID
31	cover_image	VARCHAR	500		是	封面图 URL
32	images	JSON			是	图片列表（JSON）
33	tags	VARCHAR	500		是	标签
34	province	VARCHAR	50		否	省份
35	create_time	DATETIME			是	创建时间
36	update_time	DATETIME			是	更新时间

房源图片表专门用于开展房源多媒体图片资源的管理工作，此表与房源表构建了“一对多”的关系，也就是说一套房源可以关联多张展示图片。 image_url 字段用于存储图片的完整访问路径，sort 字段则是整型排序权重，前端会依据该值从小到大来呈现图片的轮播顺序。该表与 houses 表借助外键 house_id 建立起级联删除也就是 ON DELETE CASCADE 约束，要是某套房源记录被删除，其关联的全部图片记录便会自动随之清除，以避免产生孤立数据。尽管表结构简洁，但在实际应用中却担负着房源详情页图片展示以及首页封面图渲染等重要前端功能，如表格 3-3 所示。

表格 3-3 房源图片表结构（house_images）

序号	列名	数据类型	长度	约束	允许空	说明
1	id	BIGINT		主键	否	房源图片 ID
2	house_id	BIGINT		外键	否	关联房源 ID
3	image_url	VARCHAR	500		否	图片地址
4	sort	INT		默认值	是	排序权重，（默认值为 0）
5	create_time	DATETIME			是	上传时间

订单表记录了租客以及房东之间的业务交互，把它当作租赁交易流程的数据载体来使用。其涵盖了从初步接触到最终签约的完整业务链路。order_no 字段用于保存系统自动生成的唯一编号，方便双方在沟通中引用以及进行后台的检索。status 字段负责去追踪订单的全生命周期：囊括 PENDING 待处理、APPROVED 已批准或 REJECTED 已拒绝、COMPLETED 已完成或 SIGNED 已签约以及 CANCELLED 已取消。

该表还同房源 ID、租客 ID 及房东 ID 这三个外键建立了关联，分别指向 houses 表和 users 表，这让系统能从房源、租客及房东这三个维度去开展查询与统计工作。appointment_time 字段用来记载预约时间，start_date 与 end_date 字段用于界定租赁期，monthly_rent 和 deposit 字段保存了月租金及押金，total_amount 字段则记录了合同总价。

payment_status 字段独立地开展支付状态的管理，像 UNPAID、PAID、REFUNDED，实现了同订单状态的解耦，得以支持灵活的退款流程，如表格 3-4 所示。

表格 3-4 订单表结构（orders）

序号	列名	数据类型	长度	约束	允许空	说明
1	id	BIGINT		主键	否	订单 ID
2	house_id	BIGINT		外键	否	关联房源 ID
3	tenant_id	BIGINT		外键	否	租客用户 ID
4	landlord_id	BIGINT		外键	否	房东用户 ID
5	order_no	VARCHAR	50	唯一	否	系统订单号
6	status	ENUM		默认值	是	订单状态，（默认值为 PENDING）
7	appointment_time	DATETIME			是	预约看房时间
8	start_date	DATE			是	租期开始日期
9	end_date	DATE			是	租期结束日期
10	monthly_rent	DECIMAL	10,2		是	月租金（元）
11	deposit	DECIMAL	10,2		是	押金（元）
12	total_amount	DECIMAL	12,2		是	订单总金额（元）
13	payment_status	ENUM		默认值	是	支付状态，（默认值为 UNPAID）
14	remark	TEXT			是	订单备注
15	create_time	DATETIME			是	创建时间
16	update_time	DATETIME			是	更新时间

合同表被运用在开展电子租赁合同全生命周期的管理工作上，它是本系统得以实现在线签约功能的核心数据结构。contract_no 字段负责存储系统生成的唯一合同编号，content 字段选用 LONGTEXT 类型去保存合同正文，并支持富文本格式。status 字段对合同从起草直至生效的流转路径进行了界定，即从 DRAFT 也就是草稿到 PENDING_SIGN 即待签署，再到 TENANT_SIGNED 租客已签再到 LANDLORD_SIGNED 房东已签，直至 FULLY_SIGNED 双方签署完成或 CANCELLED 已取消。tenant_signed 以及 landlord_signed 这两个布尔字段分别去标记双方的签署状态，并且配合 tenant_sign_time 以及 landlord_sign_time 来记录各方签署的精确时间。风控相关字段属于本表的一大特性，risk_level 字段存储系统对合同条款进行自动扫描后得出的风险等级即 LOW、MEDIUM、HIGH，risk_items 字段选用 JSON 格式去存储具体的合同风险条目明细，pdf_path 字段则用来记录合同 PDF 文件的存储路径，从而给签约双方提供了智能化的风险预警参考。pdf_path

字段记录合同 PDF 文件的存储路径，支持合同下载以及线下留档。该表借助 order_id 外键同订单表关联，凭借 house_id、tenant_id 以及 landlord_id 分别去关联房源以及双方用户，如表格 3-5 所示。

表格 3-5 合同表结构（contracts）

序号	列名	数据类型	长度	约束	允许空	说明
1	id	BIGINT		主键	否	合同 ID
2	contract_no	VARCHAR	50	唯一	否	合同编号
3	order_id	BIGINT		外键	否	关联订单 ID
4	house_id	BIGINT		外键	否	关联房源 ID
5	tenant_id	BIGINT		外键	否	租客用户 ID
6	landlord_id	BIGINT		外键	否	房东用户 ID
7	content	LONGTEXT			是	合同正文内容
8	status	ENUM		默认值	是	合同状态，（默认值为 DRAFT）
9	risk_level	ENUM		默认值	是	风险等级，（默认值为 LOW）
10	risk_items	JSON			是	风险条款列表（JSON）
11	tenant_signed	TINYINT		默认值	是	租客是否签署,（默认值为 0）
12	landlord_signed	TINYINT		默认值	是	房东是否签署,（默认值为 0）
13	tenant_sign_time	DATETIME			是	租客签署时间
14	landlord_sign_time	DATETIME			是	房东签署时间
15	start_date	DATE			是	租期开始日期
16	end_date	DATE			是	租期结束日期
17	monthly_rent	DECIMAL	10,2		是	月租金（元）
18	deposit	DECIMAL	10,2		是	押金（元）
19	pdf_path	VARCHAR	500		是	PDF 文件路径
20	sign_time	DATETIME			是	双方签署完成时间
21	workflow_instance_id	VARCHAR	100		是	工作流实例 ID
22	create_time	DATETIME			是	创建时间
23	update_time	DATETIME			是	更新时间

站内消息表负责开展系统内部通知和业务事件提醒工作，确保用户可以及时获知订单状态变更、合同进展以及系统公告等信息。type 字段把消息分为五类：SYSTEM 即系统通知、ORDER 即订单相关、CONTRACT 即合同相关、APPOINTMENT 即预约相关以及 REVIEW 即评价相关，前端基于此来渲染不同的图标和跳转链接。该表借助 user_id 外键去关联用户表，并设置级联删除，要是用户账号被移除了，它的历史消息就会一并清除，如表格 3-6 所示。

表格 3-6 站内消息表结构（messages）

序号	列名	数据类型	长度	约束	允许空	说明
1	id	BIGINT		主键	否	站内消息 ID
2	user_id	BIGINT		外键	否	接收用户 ID
3	title	VARCHAR	200		否	消息标题
4	content	TEXT			是	消息内容
5	type	ENUM		默认值	否	消息类型，（默认值为 SYSTEM）
6	related_id	BIGINT			是	关联业务记录 ID
7	is_read	TINYINT		默认值	是	是否已读，（默认值为 0）
8	create_time	DATETIME			是	创建时间

用户行为表作为个性化推荐算法的数据基础，它记录了每一位用户对于各个房源所进行的交互动作。behavior_type 字段运用 ENUM 类型来把三种行为进行区分，也就是 VIEW 即浏览、COLLECT 即收藏、ORDER 即下单，系统会在用户实施对应操作时自动把行为记录进行写入。score 字段则是用来对不同行为类型进行差异化权重的分配，像浏览行为权重为 1.0、收藏行为权重为 3.0、下单行为权重为 5.0，这种权重的梯度设计反映了用户对房源兴趣程度的递进关系。协同过滤推荐算法鉴于该表当中的数据来构建“用户-房源”评分矩阵：也就是先去计算用户相互之间的行为相似度，随后从相似用户的高分房源里去挖掘当前用户还没接触过的房源来当作推荐结果。此表同时也创建了 user_id 以及 house_id 这两个索引，以此来保证推荐算法在大数据量环境下的查询效率。两个外键都进行了级联删除策略的配置，要是用户或者房源被移除了，那么相关的行为记录也就自动清理掉了，如表格 3-7 所示。

表格 3-7 用户行为表结构（user_behaviors）

序号	列名	数据类型	长度	约束	允许空	说明
1	id	BIGINT		主键	否	用户行为 ID
2	user_id	BIGINT		外键	否	用户 ID
3	house_id	BIGINT		外键	否	房源 ID
4	behavior_type	ENUM		默认值	否	行为类型，（默认值为 VIEW）

续表格 3-7 用户行为表结构（user_behaviors）

序号	列名	数据类型	长度	约束	允许空	说明
5	score	DECIMAL	3,1	默认值	否	行为权重分数，行为评分(VIEW=1, COLLECT=3, ORDER=5)，（默认值为 1）
6	create_time	DATETIME			否	行为时间

房源评价表专门承载租客租赁结束后针对房源发表的评分与点评，构成了平台口碑体系的数据基础。rating 字段运用 1 至 5 的整数进行评分，5 分代表最高好评，而 1 分为最低差评；前端借助星级图标来把评分结果做直观展示。content 字段选用 TEXT 类型去保存评价正文，租客可以自由撰写关于房源环境、设施还有房东服务的体验感受。该表依靠 house_id 以及 user_id 两个外键分别实现与房源表和用户表的关联，并能选择关联 order_id 以便追溯评价源于哪笔订单。从房源维度针对评价开展汇总能计算平均分，从而辅助其他租客浏览详情时将其作为决策参考；从用户维度来查看评价历史，则能够被用来开展信用评估与行为分析。两个外键均做好了级联删除配置，在房源或用户被移除时其对应评价记录会自动被清除，以此来维持数据的一致性。如表格 3-8 所示。

表格 3-8 房源评价表结构（reviews）

序号	列名	数据类型	长度	约束	允许空	说明
1	id	BIGINT		主键	否	房源评价 ID
2	user_id	BIGINT		外键	否	评价用户 ID
3	house_id	BIGINT		外键	否	关联房源 ID
4	order_id	ENUM		外键	否	关联订单 ID
5	rating	INT		默认值	否	评分（1-5 星），（默认值为 5）
6	content	TEXT			是	评价文字内容
7	create_time	DATETIME			否	评价时间

3.10 本章小结

本章主要基于可行性、功能需求、非功能需求、业务流程建模以及数据流分析这五个不同层面，来对房屋租赁系统开展了较为全面的梳理工作。可行性分析表明，该系统在技术条件、经济成本以及操作便利性方面均具备实施基础。功能需求分析界定清楚了租客、房东以及管理员这三种角色的具体操作内容，特别是在房东类型三级标识以及五项费用强制披露方面，提出了有别于相近平台的差异化要求。非功能需求分析划定了系统在性能、安全、可靠性以及可扩展性上的基本约束。业务流程图以及数据流图从不同维度描述了系统的运行逻辑以及信息流转路径，为后续章节的系统设计提供了清晰的输入依据。

此部分从概念结构以及逻辑结构这两个维度出发，去完成了房屋租赁系统的数据库设计工作。在概念层面上，运用 E-R 图直观展示了用户、房源、订单、合同、消息、图片、

行为记录以及评价这 8 个核心实体间的语义关系，特别是“三类房东”以及“五项费用”这两项特色属性在房源实体当中的承载方式。在逻辑层面上，详细罗列了 8 张 MySQL 关系表的完整字段定义，囊括字段名、数据类型、中英文含义以及约束条件，并且凭借关联关系汇总表呈现了 14 条外键依赖。该设计方案已在实际系统当中得以落地运行，为后续的业务逻辑实现、推荐算法数据采集、合同风控规则匹配以及消息异步推送等功能奠定了稳固的数据基础。

4 系统实现

4.1 用户认证模块实现

用户认证模块承担着系统身份验证的核心重任，它主要囊括了注册、登录以及密码重置这三大基础功能。 前端选用 Element Plus 组件库来构建表单界面，依靠紫蓝渐变配色以及玻璃拟态设计来营造出现代感。 后端基于 Spring Security + JWT 来实现无状态认证，令牌有效期有 24 小时，每次请求会经由 JwtAuthenticationFilter 来解析令牌，同时把 UserDetails 对象注入 Security 上下文当中。

4.1.1 登录界面

登录页包含了“用户名/手机号与密码”这两项输入，点击登录后前端随即调用 /api/auth/login 接口，服务端校验通过就会返回 JWT 令牌，并保存进 localStorage 供后续请求来运用。 要是登录失败，接口将会反馈统一的错误信息，同时凭借前端拦截器来弹出提示，如图 4-1 所示。

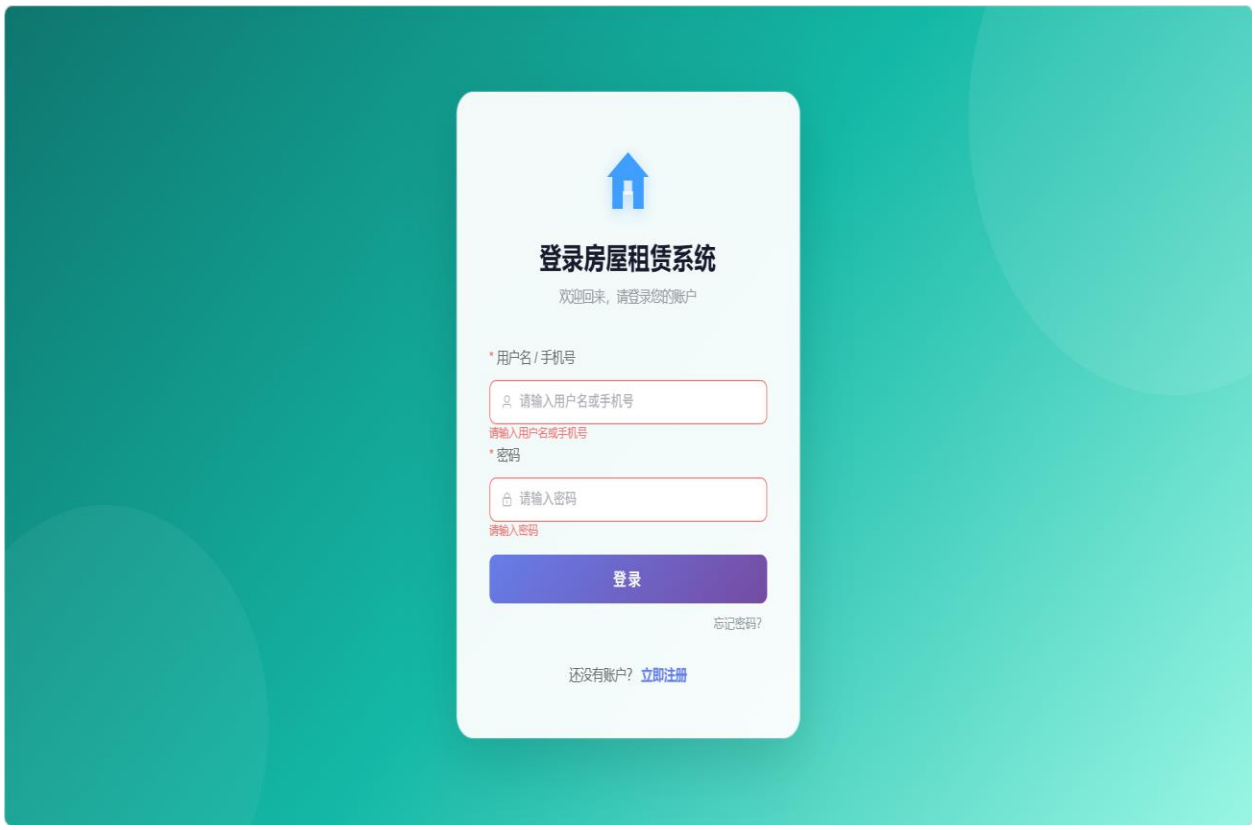

图 4-1 用户登录界面

4.1.2 注册界面

注册页面涵盖用户名、手机号、邮箱即选填项、密码以及确认密码字段，借助单选框来区分租客与房东角色，需勾选服务条款后才能提交。用户角色在注册时也就确定了，后端基于此来进行接口访问权限的控制，如图 4-2 所示。

图 4-2 用户注册界面

4.1.3 个人中心界面

个人中心涵盖了用户名、手机号、邮箱、性别、密码修改。在这当中，若要进行密码修改，需把正确的旧密码输入后，才可以修改。实名认证包含真实姓名和身份证号，如图 4-3 所示。

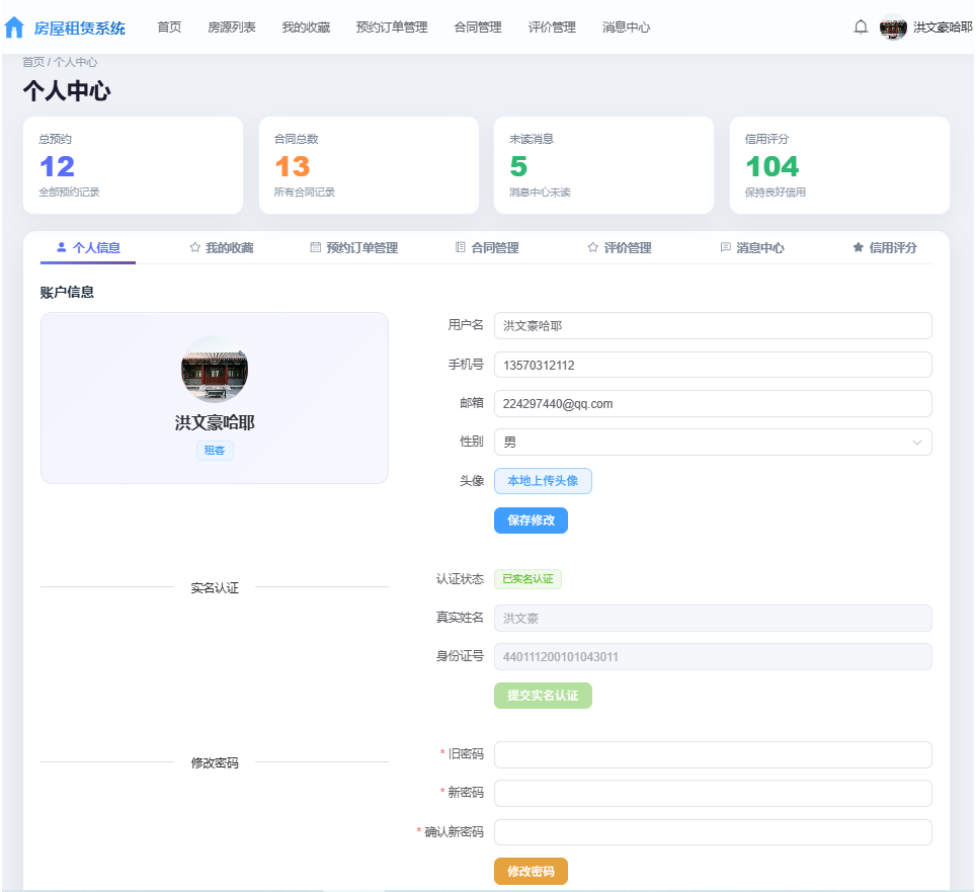

图 4-3 个人中心界面

4.1.4 认证核心代码

登录逻辑首先依靠用户名去查询用户，再运用 BCrypt 来校验密码哈希，验证通过后由 JwtUtil.generateToken() 来生成携带了用户名以及角色的 JWT 令牌，并且返回剔除了 password 以及 idCard 字段的脱敏用户信息。JwtAuthenticationFilter 在每次请求时会对令牌开展解析，从中提取用户名去加载 UserDetails，注入 Security 上下文来完成授权，后续业务借助 @AuthenticationPrincipal 来获取当前登录用户。 注册接口同步对用户名唯一性进行校验，密码选用 BCrypt 哈希来进行存储，新用户初始信用分设置为 100，状态默认为 ACTIVE，角色基于前端选择后在服务端固化，不允许后期自行提升权限。

```
//AuthController.java —— 登录接口
/**
 * 用户登录接口，验证成功后返回 JWT Token
 *
 * @param request 登录请求参数：用户名和密码
 * @return 包含 token 以及用户信息的登录结果
 */
@Operation(summary = "Login")
@PostMapping("/login")
public Result<Map<String, Object>> login(@RequestBody LoginRequest request) {
    return Result.success(userService.login(request));
}

//UserServiceImpl.java —— JWT 令牌生成
/**
 * 用户登录：验证账号密码，检查账号状态，生成 JWT Token
 *
 * @param request 登录请求参数
 * @return 包含 token 和用户信息的 Map
 */
@Override
public Map<String, Object> login(LoginRequest request) {
    // 支持「用户名」或「手机号」作为统一登录入口，减少前端区分字段的负担
    User user = userMapper.selectByUsernameOrPhone(request.getUsername());
    if (user == null) {
        throw new BusinessException(401, "用户不存在，请确定用户名或手机号是否正确
");
```

```
    }
    // 验证密码是否匹配
    if (!passwordEncoder.matches(request.getPassword(), user.getPassword())) {
        throw new BusinessException(401, "密码错误，请重新输入");
    }
    // 检查账号是否被封禁
    if ("BANNED".equals(user.getStatus())) {
        throw new BusinessException(403, "账号已被禁用");
    }
    // 每日登录信用分 +1（同一自然日仅加一次）：
    // 使用 Redis SETNX + 日期维度 key 去重，key 过期时间固定 1 天。
    String loginCreditKey = "credit:login:" + user.getId() + ":" + LocalDate.now();
    Boolean firstLoginToday = redisTemplate.opsForValue().setIfAbsent(loginCreditKey, "1",
    1, TimeUnit.DAYS);
    if (Boolean.TRUE.equals(firstLoginToday)) {
        int currentScore = user.getCreditScore() == null ? 0 : user.getCreditScore();
        user.setCreditScore(Math.min(CREDIT_SCORE_MAX, currentScore +
    DAILY_LOGIN_CREDIT_BONUS));
        user.setUpdateTime(LocalDateTime.now());
        userMapper.updateById(user);
        messageProducer.sendLoginNotification(user.getId(),
                String.format(DAILY_LOGIN_BONUS_MESSAGE_TEMPLATE,
    DAILY_LOGIN_CREDIT_BONUS));
    } else {
        messageProducer.sendLoginNotification(user.getId(),
    DAILY_LOGIN_REPEAT_MESSAGE);
    }
    // 生成 JWT Token（包含用户名和角色）
    String token = jwtUtil.generateToken(user.getId(),user.getUsername(), user.getRole());
    user.setPassword(null); // 清空密码后放入返回结果
    Map<String, Object> result = new HashMap<>();
    result.put("token", token);
    result.put("user", user);
    return result;
}
```

4.2 房源管理模块实现

房源管理属于系统的核心业务模块，它囊括了房源的搜索以及展示、发布与编辑，还有对于状态的管理工作。系统创新性地在房源信息当中强制去区分一手房东也就是 OWNER、二手房东或转租即 SUBLEASE、以及中介也就是 AGENT 这三种类型，并且要求去填写水费、电费、燃气费、物业费、网络费这五项生活成本，以此来消除传统平台的信息差。

4.2.1 房源搜索列表界面

列表页拥有关键词、省份、城市、区域、租金、房间数、房东类型以及装修情况等多维度筛选条件，同时能支持默认、价格升序及降序、最新这四种排序方式。后端搜索接口借助 MyBatis-Plus 的 LambdaQueryWrapper 来动态拼接条件，只在筛选参数非空时才追加相应过滤子句，会将热门房源结果缓存至 Redis。各筛选项基于非空这一前提来追加条件，以此规避传统 XML 进行 SQL 拼接时的繁琐与出错风险。房东类型比如 OWNER、SUBLEASE 以及 AGENT 的筛选直接对应数据库 owner_type 字段，租客能在前端借助下拉框去选择仅查看某类房源，进而直观地对房源属性进行甄别，如图 4-4 和图 4-5 所示。

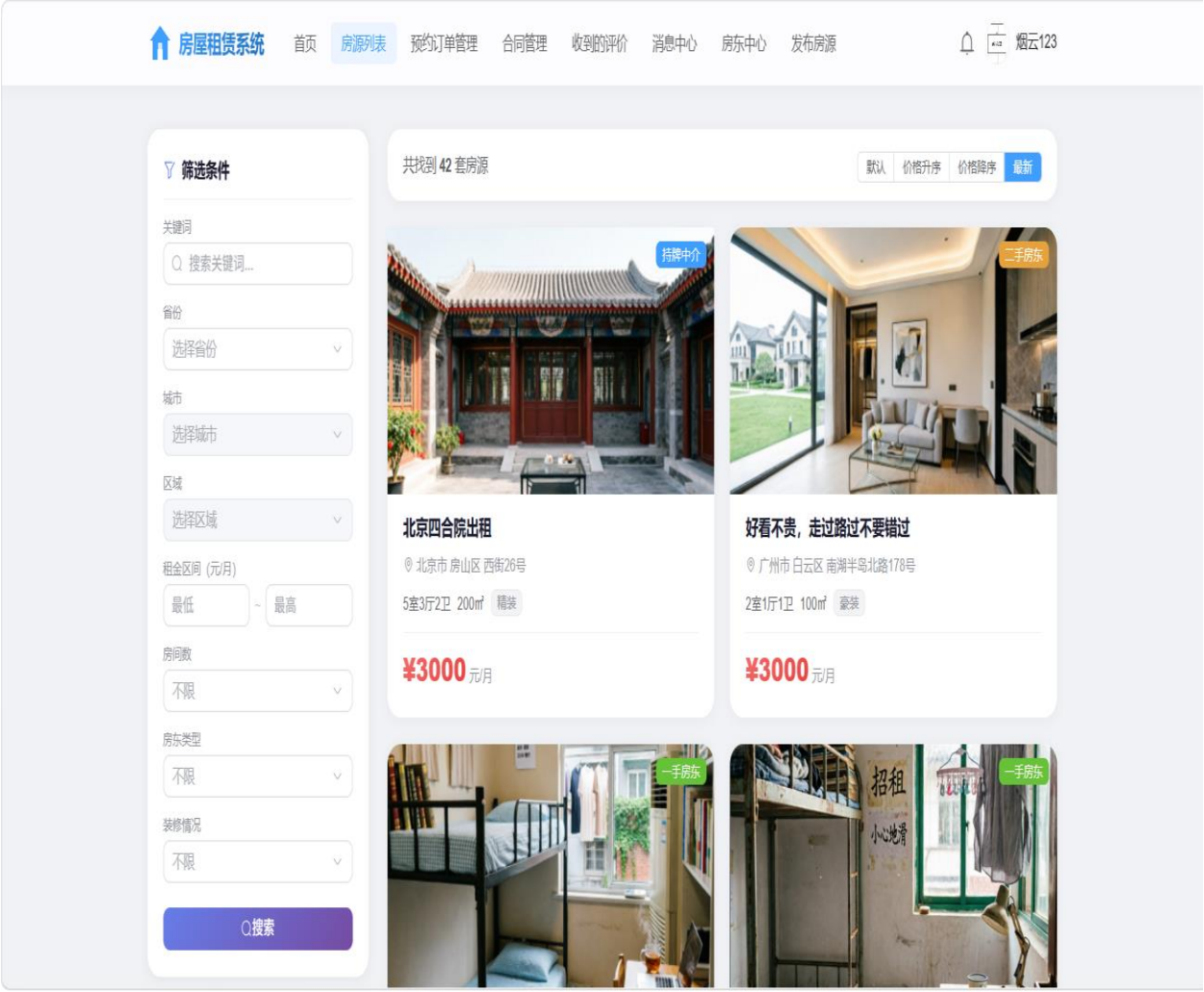

图 4-4 房源搜索列表界面

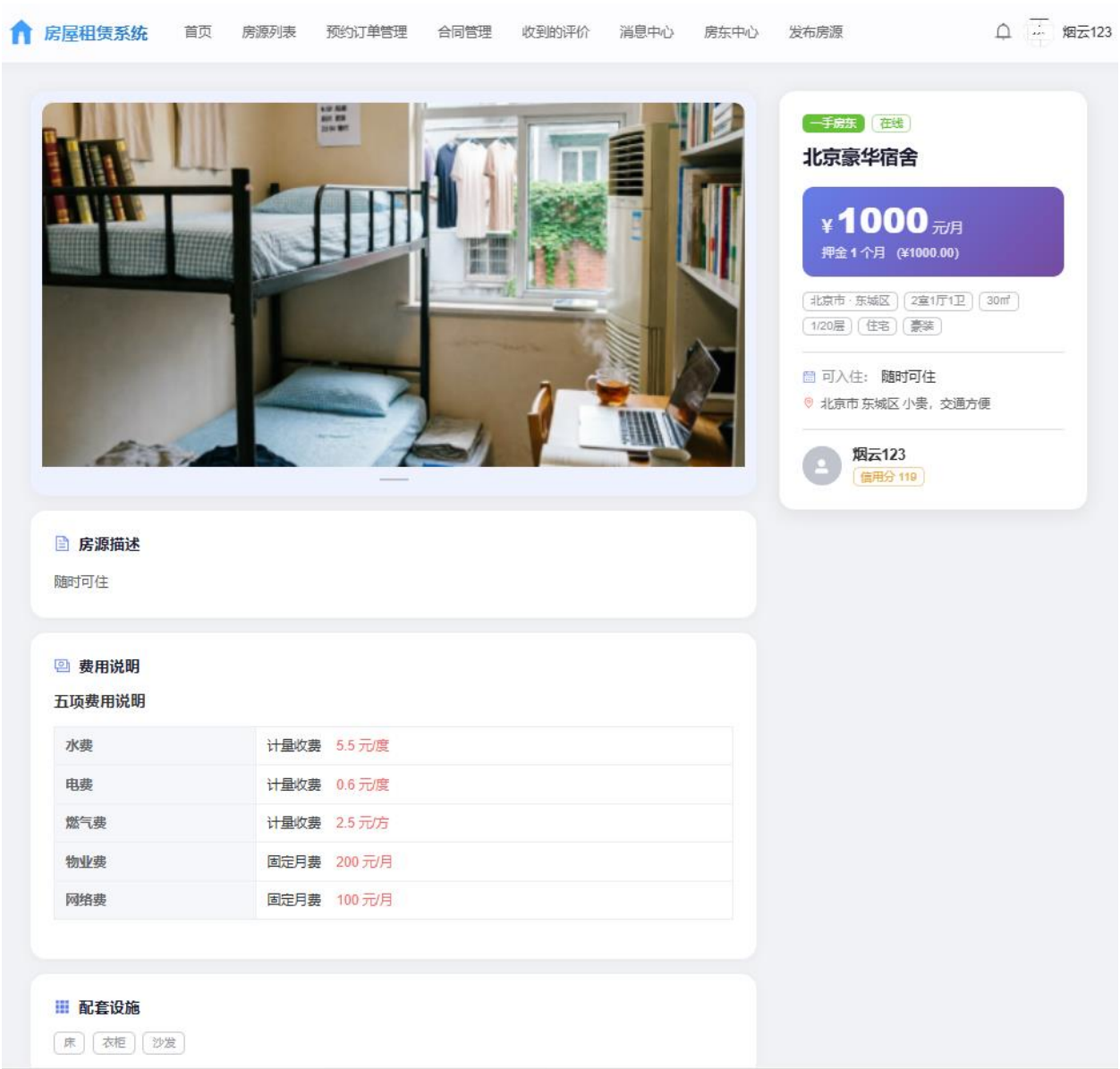

图 4-5 房源详情界面

4.2.2 Redis 缓存热门房源核心代码

热门房源借助 @Cacheable 注解把结果缓存到 Redis。每当有新房源发布或者房源信息更新时，@CacheEvict 会自动把所有热门房源缓存进行清除来保证一致性。

```
//HouseServiceImpl.java —— 热门房源 Redis 缓存
/**
 * 获取热门房源列表（按浏览量降序，最多 10 条），结果缓存到 Redis
 *
 * @return 热门房源列表
 */
@Override
@Cacheable(value = "hotHouses", key = "'all'") // 缓存热门房源列表，key 为 'hotHouses::all'
```

```
public List<House> getHotHouses() {
    try {
        LambdaQueryWrapper<House> wrapper = new LambdaQueryWrapper<>();
        wrapper.eq(House::getStatus, "ONLINE");
        wrapper.orderByDesc(House::getViewCount);
        Page<House> page = new Page<>(1, 10);
        return houseMapper.selectPage(page, wrapper).getRecords();
    } catch (Exception e) {
        log.error("Failed to get hot houses: {}", e.getMessage());
        return List.of();
    }
}

/**
 * 发布新房源，设置初始状态为已上架，清除热门房源缓存
 *
 * @param house     房源信息
 * @param ownerId  房东用户 ID
 * @return  创建成功的房源对象
 */
@Override
@Transactional
@CacheEvict(value = "hotHouses", allEntries = true) // 发布新房源后清除热门房源缓存
public House addHouse(House house, Long ownerId) {
    User owner = userMapper.selectById(ownerId);
    if (owner == null) {
        throw new BusinessException(404, "用户不存在");
    }
    if (owner.getCreditScore() != null && owner.getCreditScore() <
CREDIT_SCORE_PUBLISHING_THRESHOLD) {
        throw new BusinessException(403, "当前信用分过低，暂不可发布房源");
    }
    house.setOwnerId(ownerId);
    house.setStatus(HOUSE_STATUS_ONLINE); // 新房源默认状态为已上架
    house.setViewCount(0);
```

```
house.setCreateTime(LocalDateTime.now());
house.setUpdateTime(LocalDateTime.now());
houseMapper.insert(house);
// 同步写入 house_images 明细表，确保“房源主表 JSON 字段”和“图片明细表”两处数据一致
syncHouseImages(house.getId(), house.getImages());
return house;
}
```

4.3 首页与智能推荐模块实现

系统首页用于展示平台整体的统计数据，比如在租房源数、注册用户数、成交数量以及覆盖城市等，同时还会展示精选的推荐房源卡片，并且对三大平台优势进行说明。推荐模块是基于协同过滤算法来开发的，它能够去分析用户的历史行为，从而来为每一位用户生成个性化的房源推荐，如图 4-6 所示。

图 4-6 系统首页界面（含统计数据与推荐房源）

4.3.1 协同过滤推荐算法

该算法的核心思路鉴于“相似用户所青睐的房源，您或许也有可能感兴趣”这一逻辑来构建。它首先借助 user_behaviors 表当中的浏览、收藏以及下单记录来开展分析，进而去识别出行为重叠度较高的相似用户群体，随后统计这些用户曾浏览过但当前用户尚未接触的房源频次，频次越高就意味着推荐价值越大。要是推荐结果数量不足，系统会自动采用按浏览量排序的热门房源来进行填充，同时异常路径也会降级成为热门房源推荐，以此来确保接口的可用性。

```
//RecommendationServiceImpl.java —— 基于用户行为的协同过滤
/**
 * 基于协同过滤为用户推荐房源
 * 算法分五步执行，异常时降级为热门房源推荐
 *
 * @param userId 用户 ID
 * @param limit  最多返回的推荐数量
 * @return 推荐的房源列表
 */
@Override
public List<House> getRecommendedHouses(Long userId, int limit) {
    try {
        // 第一步：获取当前用户有过交互的所有房源集合
        List<Long> userHouseIds = userBehaviorMapper.selectHouseIdsByUserId(userId);
        Set<Long> userHouseSet = new HashSet<>(userHouseIds);
        // 第二步：找到与当前用户行为相似的其他用户（基于共同感兴趣的房源>=1）
        Set<Long> similarUsers = new HashSet<>();
        for (Long houseId : userHouseIds) {
            List<Long> otherUsers = userBehaviorMapper.selectUserIdsByHouseId(houseI
d, userId);
            similarUsers.addAll(otherUsers);
        }
        // 第三步：统计相似用户感兴趣但当前用户未看过的房源的出现频次
        Map<Long, Integer> houseFrequency = new HashMap<>();
        for (Long similarUserId : similarUsers) {
            List<Long> similarUserHouses = userBehaviorMapper.selectHouseIdsByUserI
d(similarUserId);
            for (Long houseId : similarUserHouses) {
```

```
                if (!userHouseSet.contains(houseId)) {
                    // 相似用户每次出现该房源就计数 +1
                    houseFrequency.merge(houseId, 1, Integer::sum);
                }
            }
        }
        // 第四步：按频次降序排列，取前 limit 条房源 ID
        List<Long> recommendedIds = houseFrequency.entrySet().stream()
                .sorted(Map.Entry.<Long, Integer>comparingByValue().reversed())
                .limit(limit)
                .map(Map.Entry::getKey)
                .collect(Collectors.toList());

        List<House> recommended = new ArrayList<>();
        if (!recommendedIds.isEmpty()) {
            // 批量查询推荐房源，只返回已上线的房源
            LambdaQueryWrapper<House> wrapper = new LambdaQueryWrapper<>();
            wrapper.in(House::getId, recommendedIds)
                    .eq(House::getStatus, "ONLINE");
            recommended = houseMapper.selectList(wrapper);
        }//...（查询并返回，降级保底：异常时直接返回热门房源）
```

4.4 合同管理与风险识别模块实现

合同管理模块能基于已审批订单来完成租赁合同文本自动生成、电子签署、PDF 导出以及全生命周期状态的追踪。系统会在生成合同时同步触发风险分析服务，凭借预设规则库对合同文本进行多维度扫描，来识别押金超标、条款缺失等风险点，并且选用 HIGH/MEDIUM/LOW 三级来标注，不需要依赖复杂的 NLP 模型就能实现对常见风险场景的有效覆盖。

4.4.1 合同风险识别规则

合同风险识别规则，如表格 4-1 所示。

表格 4-1 合同风控规则库

规则号	风险类型	检测逻辑	风险级别
R-01	押金过高	押金>月租金×2	HIGH
R-02	维修条款缺失	合同文本中不含“维修”或“修缮”	MEDIUM

续表格 4-1 合同风控规则库

规则号	风险类型	检测逻辑	风险级别
R-03	解约条款缺失	不含“提前解约”或“提前终止”	MEDIUM
R-04	违约金过高	违约金条款中出现“4 个月”/“5 个月”租金	HIGH
R-04	费用未约定	不含“水费”且不含“电费”	LOW

4.4.2 风控服务核心代码

规则引擎运用顺序扫描的形式逐条开展评估，每条规则独立触发并且互不干扰，这样便于后续在不修改已有逻辑的基础上去追加新规则。目前五条规则主要囊括押金比例、维修条款、提前解约、违约金额度以及水电费约定，一旦命中规则后会把 RiskItem 也就是 title、description、level 的对象加入列表。风险结果按照 JSON 格式进行序列化并且保存到 contracts 表的 risk_items 字段也就是 JSON 列类型，前端合同详情页进行反序列化后借助彩色标签来呈现，比如红色 HIGH、橙色 MEDIUM、绿色 LOW，进而帮助租客以及房东在签署前直观地了解合同潜在风险，降低纠纷概率。

```
//ContractRiskServiceImpl.java —— 规则引擎风险分析即 5 条规则（5 条规则）
/**
 * 对合同文本开展多维度风险分析
 * 依次检查五类风险：押金比例、维修条款、提前解约、违约金额度、水电费约定
 *
 * @param contractText  合同正文文本
 * @param monthlyRent   月租金（用于计算押金倍数）
 * @param deposit       押金金额
 * @return  发现的风险条目列表
 */
@Override
public List<RiskItem> analyzeRisk(String contractText, BigDecimal monthlyRent, BigDecimal
deposit) {
    List<RiskItem> risks = new ArrayList<>();
    if (contractText == null) return risks;

    // 风险一：检查押金是否超过两个月租金
    if (monthlyRent != null && deposit != null) {
        if (deposit.compareTo(monthlyRent.multiply(new BigDecimal("2"))) > 0) {
```

```
                risks.add(new RiskItem("押金过高", "押金超过两个月租金，请注意资金安全",
"HIGH"));
            }
        }

        // 风险二：检查是否包含维修责任条款
        if (!contractText.contains("维修") && !contractText.contains("修缮")) {
            risks.add(new RiskItem("条款缺失", "合同未约定维修责任，建议补充相关条款",
"MEDIUM"));
        }

        // 风险三：检查是否包含提前解约条款
        if (!contractText.contains("提前解约") && !contractText.contains("提前终止")) {
            risks.add(new RiskItem("条款缺失", "合同未约定提前解约条款，建议明确违约处
理方式", "MEDIUM"));
        }

        // 风险四：检查违约金是否过高（简单启发式：查找含"4 个月"/"5 个月"违约金的描述）
        if (contractText.contains("违约金") && monthlyRent != null) {
            // 简单启发规则：如果违约金附近出现 4 个月或 5 个月的描述，则标记为高风险
            if (contractText.contains("违约金") &&
                    (contractText.contains("4 个月") || contractText.contains("5 个月") ||
                     contractText.contains("四个月") || contractText.contains("五个月"))) {
                risks.add(new RiskItem("违约金过高", "违约金条款可能超过 3 个月租金，请
仔细核查", "HIGH"));
            }
        }

        // 风险五：检查水电费标准是否有明确约定
        if (!contractText.contains("水费") && !contractText.contains("电费")) {
            risks.add(new RiskItem("费用未约定", "合同未约定水电费标准，建议明确收费标
准", "LOW"));
        }
        return risks;
    }
```

4.4.3 合同生成与签署接口

在生成合同的时候，服务层会自动把订单里的用户 ID、房源、租金租期这些关键数据提取出来，填进合同模板，并且马上调用风控服务去生成 risk_items 列表，凭借一次接口调用便完成合同创建工作以及风险预警。签署流程能够支持租客以及房东分别进行操作：也就是 tenantSigned 以及 landlordSigned 字段独立记录状态，activiti 只能租客签署完合同后，才到房东签署合同；双方签署后状态会自动从 DRAFT 最终变为 FULLY_SIGNED，同时借助 RabbitMQ 异步推送生效通知，如果 RabbitMQ 不可用就降级为同步写入 messages 表，以此确保状态变更在业务及消息层之间保持一致，如图 4-7 所示。

图 4-7 合同界面

//ContractController.java —— 合同生成与电子签署

```
/**
 * 根据订单生成租赁合同（包含自动风险分析）
 *
 * @param request        合同生成请求（订单 ID 和补充条款）
 * @param userDetails  当前登录用户信息
 * @return  生成的合同对象
 */
@Operation(summary = "Generate contract from order")
@PostMapping("/generate")
public Result<Contract> generateContract(@RequestBody ContractGenerateRequest request,
                                         @AuthenticationPrincipal UserDetails
userDetails) {
    User user = resolveUser(userDetails.getUsername());
// 服务层：自动填充双方信息、租期、金额，同步触发风险分析
    return Result.success(contractService.generateContract(request, user.getId()));
}

/**
 * 用户对合同进行电子签署
 *
 * @param id              合同 ID
 * @param request        请求体，包含 role 字段（TENANT 或 LANDLORD）
 * @param userDetails  当前登录用户信息
 * @return  签署后的合同对象
 */
@Operation(summary = "Sign contract")
@PostMapping("/{id}/sign")
public Result<Contract> signContract(@PathVariable Long id,
                                     @RequestBody Map<String, String> request,
                                     @AuthenticationPrincipal UserDetails
userDetails) {
    User user = resolveUser(userDetails.getUsername());
    String role = request.get("role");// TENANT 或 LANDLORD
```

```
// 双方均签署后合同状态自动变更为生效
    return Result.success(contractService.signContract(id, user.getId(), role));
}
```

4.5 消息通知模块实现

消息通知模块借助 RabbitMQ 消息队列来实现异步推送，把通知逻辑和主业务流程进行解耦，以免出现同步调用阻塞影响接口响应速度的情况。系统囊括了预约确认、合同状态变更以及订单状态变更这三类通知场景，所有的消息都会凭借 house.exchange 交换机路由到对应的队列当中，随后由消费者开展异步处理工作。

MessageProducer 运用@Autowired(required=false)来可选注入 RabbitTemplate，这能让系统在没有部署 RabbitMQ 的时候也可以正常运行——也就是在此时会自动降级并直接去调用 MessageService，把通知写入 messages 表中。这个设计得以实现“有 MQ 的时候就异步、没 MQ 的时候就同步”这样的弹性架构，降低了环境依赖方面的风险，比较适宜那种从单机部署逐步扩展到分布式环境的迭代场景。

```
//MessageProducer.java —— RabbitMQ 异步推送（含降级保底）
/**
 * 发送合同状态变更通知消息
 * 通知用户合同的最新状态（如已签署、已取消等）
 *
 * @param userId            接收通知的用户 ID
 * @param contractStatus  合同状态描述
 */
public void sendContractStatusChange(Long userId, String contractStatus,Long relatedId) {
    String content = "合同状态更新：" + contractStatus;
    try {
        if (rabbitTemplate != null) {
            // RabbitMQ 可用：发送到队列异步处理
            rabbitTemplate.convertAndSend("house.exchange", "contract.status",
                    buildMessage(userId, "合同状态通知", content,relatedId));
            log.debug("已把该用户的合约状态变更请求放入消息队列，等待处理 {}",
userId);
        } else {
            saveMessageDirectly(userId, "合同状态通知", content, "CONTRACT",
relatedId);
        }
```

```
    } catch (Exception e) {
        log.warn("鉴于 RabbitMQ 服务不可用，系统已将消息暂存至本地（或数据库），等待后续处理: {}", e.getMessage());
        saveMessageDirectly(userId, "合同状态通知", content, "CONTRACT", relatedId);
    }
}
```

4.6 后台管理模块实现

后台管理模块面向 ADMIN 角色，拥有用户管理、房源管理、预约订单管理、合同管理以及数据概览与可视化等功能。管理员能在后台去查看平台运营数据，并借助 ECharts 图表来展示房源区域数量以及价格走势等指标，也能对房东上传的房源开展上架或者下架操作，来实现对平台内容的精细管控，如图 4-8 所示。

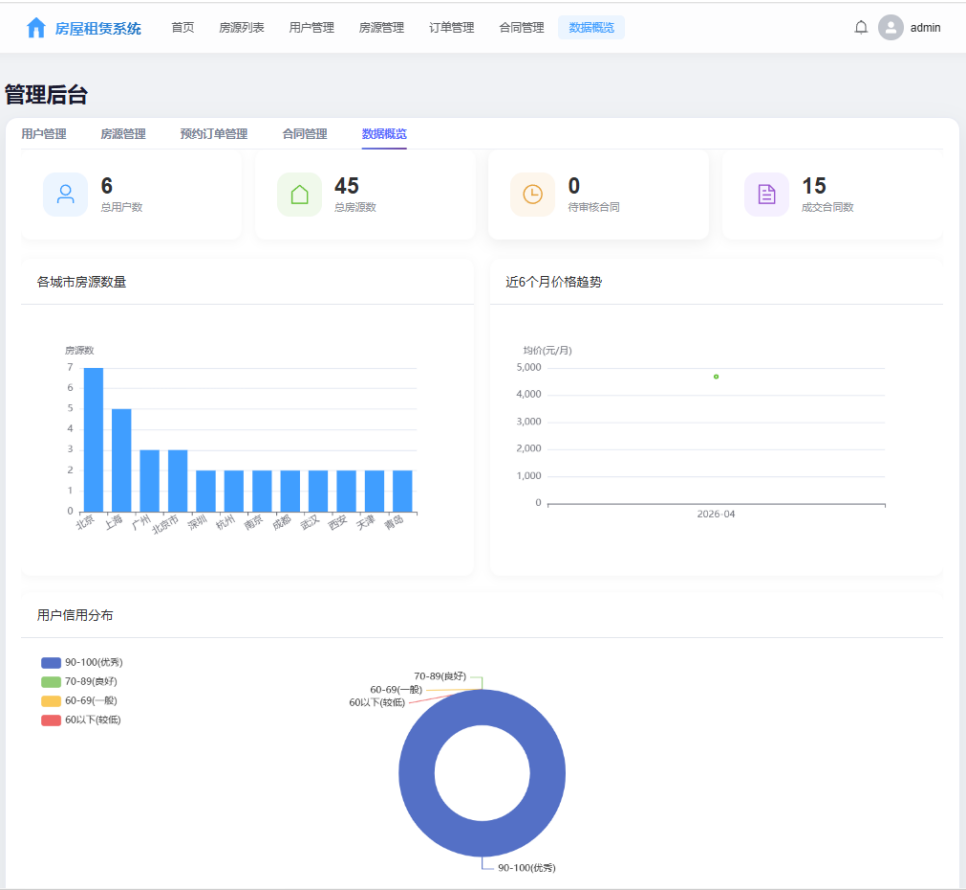

图 4-8 管理后台界面

统计核心代码：

```
//AdminController.java —— 平台统计
/**
 * 获取系统统计数据（用户总数、房源总数、订单总数、合同总数）
```

```
 *
 * @return 包含各项统计数据的 Map
 */
@Operation(summary = "Get system statistics")
@GetMapping("/statistics")
public Result<Map<String, Object>> getStatistics() {
    Map<String, Object> stats = new HashMap<>();
    stats.put("userCount", userMapper.selectCount(null));        // 用户总数
    stats.put("houseCount", houseMapper.selectCount(null));     // 房源总数
    stats.put("orderCount", orderMapper.selectCount(null));     // 订单总数
    // 合同统计：总数及待签署/审核数量
    stats.put("contractCount", contractMapper.selectCount(null)); // 合同总数
    com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper<Contract>
pendingWrapper =
            new
com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper<>();
    pendingWrapper.eq(Contract::getStatus, "PENDING_SIGN");
    stats.put("pendingContracts", contractMapper.selectCount(pendingWrapper)); // 待审核合
同数
    return Result.success(stats);
}
```

平台统计接口借助 MyBatis-Plus 开展聚合查询来获取实时数据，前端进行周期性轮询后渲染为可视化图表。房源管理支持 ONLINE 即已上架、OFFLINE 即已下架两种状态的操作，管理员审核后，如房源状态由 ONLINE 变更为 OFFLINE，系统同时借助消息队列去向房东发送下架通知。

4.7 Activiti 流程引擎集成

系统集成了 Activiti 7 工作流引擎，合同签署等关键业务节点凭借 BPMN 流程定义，依靠引擎驱动节点状态流转，让审批过程有据可查、得以追溯。较之纯状态机实现，流程引擎引入进一步降低了业务扩展时需去手动维护状态跳转逻辑的成本。

合同签署审核流程核心代码：

当需要发起一份新合同的签署时，调用此方法。它会向流程引擎注册一个新的流程实例（contractSigningProcess），并传入合同 ID、租客 ID、房东 ID 等变量。

返回流程实例 ID，后续所有与该合同签署相关的流程操作都通过此 ID 进行。

//WorkflowService.java ——合同签署工作流程

```
/**
 * 启动合同签署流程
 *
 * @param contractId 合同 ID
 * @param tenantId   租客 ID
 * @param landlordId 房东 ID
 * @return 流程实例 ID
 */
public String startContractSigningProcess(Long contractId, Long tenantId, Long landlordId) {
    Map<String, Object> variables = new HashMap<>();
    // 组装流程变量，供后续用户任务节点读取使用
    variables.put("contractId", contractId);
    variables.put("tenantId", tenantId);
    variables.put("landlordId", landlordId);
    // 启动流程，并设置业务主键为 CONTRACT-{id}，方便后续关联查询
    ProcessInstance instance = runtimeService.startProcessInstanceByKey(
            "contractSigningProcess",
            "CONTRACT-" + contractId,
            variables);
    log.info("Started contractSigningProcess instance {} for contract {}",
instance.getProcessInstanceId(), contractId);
    return instance.getProcessInstanceId();
}
```

租客或房东在前端完成签署操作后，调用此方法。它会查找当前用户在该流程实例下的待办任务，并根据角色（TENANT 或 LANDLORD）设置相应的流程变量（tenantSigned 或 landlordSigned）。

任务完成后，流程引擎根据流程变量和网关判断流程走向（如双方都同意则流程结束，否则可能驳回或终止）。

```
// 合同签署（是否同意签署）
/**
 * 完成合同签署任务
 *
 * @param processInstanceId 流程实例 ID
```

```
 * @param userId            当前签署人 ID
 * @param role              签署角色 TENANT / LANDLORD
 * @param approved          是否同意签署
 */
public void completeContractTask(String processInstanceId, Long userId, String role, boolean
approved) {
    // 1. 查询待办任务
    // 构建任务查询条件：匹配指定的流程实例 ID 以及当前用户 ID（需转为 String 以匹
配 Assignee 字段）
    // singleResult() 确保查询结果唯一，如果无结果返回 null，多于一个结果则抛出异常
    Task task = taskService.createTaskQuery()
            .processInstanceId(processInstanceId)
            .taskAssignee(String.valueOf(userId))
            .singleResult();

    // 2. 校验任务是否存在
    // 如果查询结果为空，说明当前用户在该流程中没有待处理的签署任务，抛出业务异
常
    if (task == null) {
        throw new BusinessException(404, "未找到合同签署任务");
    }

    // 3. 准备流程变量
    // 用于在完成任务时回写给工作流引擎，驱动流程流转或更新流程状态
    Map<String, Object> vars = new HashMap<>();
    // 设置通用的审批结果变量，供流程网关判断走向（如：通过则进入下一节点，拒绝
则结束或驳回）
    vars.put("approved", approved);

    // 4. 根据角色设置特定的签署状态变量
    // 这种设计允许流程中记录每一方的独立签署状态，便于后续查询或并行签署逻辑
    if ("TENANT".equals(role)) {
        vars.put("tenantSigned", approved);
        // 如果是租客角色，设置租客已签署变量
    } else if ("LANDLORD".equals(role)) {
```

```
            vars.put("landlordSigned", approved);
        }

        // 5. 完成任务
        // 调用工作流服务，传入任务 ID 和流程变量集合，正式结束当前任务节点
        taskService.complete(task.getId(), vars);

        // 6. 记录操作日志
        // 记录用户 ID、任务 ID 和角色，便于后续审计和问题追踪
        log.info("User {} completed contract task {} with role {}", userId, task.getId(), role);
}
```

判断合同签署流程是否已结束。通过查询运行时和历史流程实例，若流程已结束则返回 true。

```
// 判断流程是否已经结束
/**
 * 判断流程是否已经结束
 *
 * @param processInstanceId 流程实例 ID
 * @return true 表示已结束
 */
public boolean isProcessFinished(String processInstanceId) {
        ProcessInstance runtime = runtimeService.createProcessInstanceQuery()
                .processInstanceId(processInstanceId)
                .singleResult();
        if (runtime != null) {
            return false;
        }
        HistoricProcessInstance history = historyService.createHistoricProcessInstanceQuery()
                .processInstanceId(processInstanceId)
                .finished()
                .singleResult();
        return history != null;
}
```

5 系统测试

5.1 测试目的

本次测试主要为了验证房屋租赁系统各模块是否基于需求说明书来运行。具体目标包括：第一，去核实用户注册、登录及角色切换这类认证功能的正确性与健壮性；第二，针对房源发布、编辑、管理上下线等管理流程的完整性开展检验，并核对三类房东标识与五项费用明细等创新功能展示的准确性；第三，针对租赁意向提交、房东审批及订单状态流转等业务逻辑来开展测试；第四，去验证管理员进行房源管理、用户管理等后台操作的有效性；第五，针对 JWT 身份认证、角色权限控制、参数校验及 SQL 注入防护等安全机制的可靠性来开展评估。

5.2 测试资源

Window10 专业版 64 位、Google Chrome 浏览器、JDK17、MySQL8.0、Postman。

5.3 测试方法

这次测试综合选用了黑盒测试以及接口测试这两种主要的方法。黑盒测试是站在终端用户的角度来开展的，会借助操作系统的界面来验证各个页面功能的输入输出是不是契合预期的结果，同时也会运用等价类划分以及边界值分析的技术，把手机号、密码还有租金金额这些输入域划分成有效等价类与无效等价类，并选取那些具有代表性的测试数据来进行覆盖。接口测试则是借助 Postman 这个工具直接去调用后端的 RESTful API，以此来验证请求参数的校验、JWT 令牌的鉴权、角色权限的控制以及异常响应处理的正确性。

5.4 测试用例及执行

根据系统功能划分，本节从用户认证、房源管理、租赁业务、后台管理和安全性五个模块分别设计测试用例，共计 25 条。

5.4.1 用户认证模块测试用例

根据用户认证模块覆盖注册、登录及身份校验功能，测试用例如表格 5-1 所示。

表格 5-1 用户认证模块测试用例

序号	用例名称	步骤	预期结果	实际结果	测试结论
1	合法信息注册	填写用户名、手机号、密码提交。	注册成功，跳转登录页。	注册成功，跳转登录页。	测试通过
2	重复用户名注册	使用已存在用户名注册。	提示“用户名已被注册”。	提示“用户名已被注册”。	测试通过
3	手机号格式校验	输入 8 位非法手机号注册。	前端表单校验不通过。	前端表单校验不通过。	测试通过

续表格 5-1 用户认证模块测试用例

序号	用例名称	步骤	预期结果	实际结果	测试结论
4	正确账号密码登录	输入已注册账号密码。	登录成功，导航栏显示用户名。	登录成功，导航栏显示用户名。	测试通过
5	错误密码登录	输入错误密码提交	提示“用户名或密码错误”。	提示“用户名或密码错误”。	测试通过

5.4.2 房源管理模块测试用例

房源管理模块涵盖房源发布、信息展示与搜索筛选功能，测试用例如表格 5-2 所示。

表格 5-2 房源管理模块测试用例

序号	用例名称	步骤	预期结果	实际结果	测试结论
6	完整信息发布房源	房东填写全部必填字段并上传图片提交。	发布成功，房源状态为已上架。	发布成功，房源状态为已上架。	测试通过
7	必填字段缺失发布	不填写地址直接提交。	前端提示“请填写详细地址”。	前端提示“请填写详细地址”。	测试通过
8	三类房东标签展示	查看不同房东类型的房源卡片。	分别显示业主直租/转租/中介对应颜色标签。	分别显示业主直租/转租/中介对应颜色标签。	测试通过
9	五项费用明细展示	进入房源详情页。	完整显示水电燃气物业网络五项费用信息。	完整显示水电燃气物业网络五项费用信息。	测试通过
10	多条件组合搜索	按城市+价格区间+户型搜索。	仅返回符合全部条件的房源列表。	仅返回符合全部条件的房源列表。	测试通过

5.4.3 租赁业务模块测试用例

租赁业务模块覆盖下单、审批、合同生成与签署等核心流程，测试用例如表格 5-3 所示。

表格 5-3 租赁业务模块测试用例

序号	用例名称	步骤	预期结果	实际结果	测试结论
11	租客提交预约看房	选择房源并填写预约日期和留言提交。	订单创建成功，状态为 PENDING。	订单创建成功，状态为 PENDING。	测试通过
12	房东审批通过	房东在预约列表点击“确认预约”。	状态变为 APPROVED，租客收到通知。	状态变为 APPROVED，租客收到通知。	测试通过
13	房东拒绝预约	房东点击“拒绝”。	状态变为 REJECTED，租客可查看拒绝原因。	状态变为 REJECTED，租客可查看拒绝原因。	测试通过
14	租客取消预约	租客对 PENDING 状态订单点击“取消”。	状态变为 CANCELLED。	状态变为 CANCELLED。	测试通过
15	生成租赁合同	基于 APPROVED 订单生成合同。	合同生成成功，包含风险检测结果。	合同生成成功，包含风险检测结果。	测试通过
16	双方签署合同	租客与房东依次确认签署。	状态流转至 FULLY_SIGNED。	状态流转至 FULLY_SIGNED。	测试通过
17	合同 PDF 导出	点击“导出 PDF”按钮。	浏览器下载完整 PDF 合同文件。	浏览器下载完整 PDF 合同文件。	测试通过

5.4.4 后台管理模块测试用例

后台管理模块包含管理员对房源、用户的审核与管理操作，测试用例如表格 5-4 所示。

表格 5-4 后台管理模块测试用例

序号	用例名称	步骤	预期结果	实际结果	测试结论
18	通过房源	管理员点击“上架”。	房源状态变为 ONLINE，前台可见。	房源状态变为 ONLINE，前台可见。	测试通过
19	下架房源	管理员点击“下架”并填写原因。	房源状态变为 OFFLINE。	房源状态变为 OFFLINE。	测试通过

续表格 5-4 后台管理模块测试用例

序号	用例名称	步骤	预期结果	实际结果	测试结论
20	封禁违规用户	管理员将用户状态设为 BANNED。	该用户无法登录，提示账号已被禁用。	该用户无法登录，提示账号已被禁用。	测试通过
21	查看平台统计数据	管理员进入仪表盘页面。	正确显示房源数、用户数、订单数等统计。	正确显示房源数、用户数、订单数等统计。	测试通过

5.4.5 安全性测试用例

安全性测试针对身份认证、权限控制和输入防护三个维度，测试用例如表格 5-5 所示。

表格 5-5 安全性测试用例

序号	用例名称	步骤	预期结果	实际结果	测试结论
22	过期 JWT 令牌访问	携带过期 Token 请求受保护接口。	返回 401 未授权状态码。	返回 401 未授权状态码。	测试通过
23	普通用户访问管理接口	租客角色调用/admin/*接口。	返回 403 禁止访问。	返回 403 禁止访问。	测试通过
24	SQL 注入攻击	搜索框输入' OR 1=1 --。	正常返回空结果，无数据泄露。	正常返回空结果，无数据泄露。	测试通过
25	押金异常值风险检测	合同押金设置为月租金 5 倍。	系统提示“押金超出常规范围”风险项。	系统提示“押金超出常规范围”风险项。	测试通过

5.5 测试报告

此次测试开展了 25 条用例的执行，囊括用户认证、房源管理、租赁业务、后台管理以及安全性五大模块。首轮测试里 3 条用例没能一次性通过，经排查全属于实现层面的细节缺陷，如表格 5-6 所示。

表格 5-6 缺陷记录与修复

缺陷序号	所属模块	问题描述	修复措施	状态
BUG-1	房源管理	房源面积字段允许输入负值。	后端增加@Min(1)参数校验注解。	已修复
BUG-2	租赁业务	合同 PDF 导出中文偶发乱码。	PDF 生成时指定中文字体资源。	已修复
BUG-3	SQL 注入	搜索框输入' OR 1=1 --。	正常返回空结果，无数据泄露。	已修复

经修复后进行全量回归测试，25 条用例全部通过，通过率为 100%。各模块测试用例数量与通过情况如表格 5-7 所示。

表格 5-7 测试结果统计

测试模块	用例数量	通过数量	通过率
用户认证模块	5	5	100%
房源管理模块	5	5	100%
租赁业务模块	7	7	100%
后台管理模块	4	4	100%
安全性测试	4	4	100%
合计	25	25	100%

综合测试结果表明，房屋租赁系统各核心业务流程运行准确，用户认证以及角色权限机制是有效的，三类房东标识以及五项费用明细功能展示清晰，合同风险识别规则工作正常，后台进行的管理操作响应准确，系统在安全防护层面有着基础的抗攻击能力。整体质量符合需求规格说明书的设计要求，有着进行部署上线的条件。

6 总结与展望

6.1 研究工作总结

鉴于住房租赁市场当中长期存有房东身份不明、生活费用披露缺失、合同条款失范以及供需匹配低效等结构性问题，本文以此为切入点，并基于对贝壳、自如及 Zillow 等主流平台功能短板的分析，设计并开展了基于 Spring Boot 与 Vue 前后端分离架构的房屋租赁管理平台的实现工作。后端选用 Spring Boot 3.x 作为基础，整合 MyBatis-Plus 去开展业务逻辑处理及数据持久化，借助 JWT 达成无状态身份认证；前端采用 Vue 3 结合 Element Plus 搭建单页应用，依靠 Axios 与后端 RESTful 接口开展交互，并利用 Pinia 进行全局状态的管理。在中间件方面，Redis 负责热点房源缓存及每日首次登录管理；RabbitMQ 运用 Topic 模式构建预约、订单以及合同三条消息通道，实现了业务事件的异步解耦，要是中间件无法连接，系统会进行自动降级把数据写入数据库；Activiti 7 流程引擎驱动了合同签署等审批节点，使得业务流转具备可配置性与可追溯性。

6.2 主要功能与创新点

系统是针对租客、房东以及管理员这三种角色来进行功能设计的。租客端拥有多维筛选、在线签约、订单全程跟踪以及信用分查询的功能；房东端支持开展房源发布以及管理、租赁审批以及收益统计，发布时会强制录入水、电、燃气、物业、网络五项费用的计价方式以及历史均价，从信息录入源头来压缩费用不透明的问题；管理员后台集成了用户管理、房源管理、订单合同监控以及 ECharts 运营看板，能呈现区域热度、价格走势等指标，给平台运营决策提供量化依据。

系统的创新性主要体现在三个方面。第一，提出了三类房东以及五项费用双透明模型，把房东划分成一手房东、转租方以及持牌中介，在详情页用差异化标签来进行显著标注，配合五项费用强制披露机制，从制度层面来有效压缩信息不对称。第二，构建了基于协同过滤的个性化推荐引擎，持续采集用户浏览、收藏以及下单行为，去识别兴趣相近群体并且推送偏好房源，在数据稀疏时会自动切换到热度兜底策略。第三，开发了基于规则库的合同风控模块，合同生成后会自动扫描押金比例、维修责任、解约条件、违约金以及水电约定等条目并且进行分级预警，该方案不需要大型语言模型支撑，部署成本低并且响应迅速。

6.3 不足之处

本系统当中依然存在若干局限性。协同过滤算法在新用户注册初期鉴于行为数据匮乏，推荐覆盖率偏低，后续能够凭借引入注册属性标签或深度学习嵌入表示来开展改善工作。合同风控模块主要依靠预定义规则库，针对措辞复杂或刻意避开关键词的条款识别能力有限，未来能够把自然语言处理技术结合进来以提升语义理解精度。在运维层面，系统已具备 Docker 容器化基础，但完整的微服务拆分以及 CI/CD 流水线仍有待建立，像 VR 全景看房以及区块链合同存证这类的扩展功能也正处于规划阶段当中。

6.4 后续展望

后续拟定从三个方向来开展推进方面的工作。首先是在推荐侧融入用户注册画像以及轻量深度模型，以此来对冷启动问题进行缓解；其次是在风控侧引入自然语言处理技术，提高对非标条款的语义识别能力；最后是去完善微服务架构以及自动化部署流水线，并且逐步探索区块链合同存证、VR 全景看房以及智能定价辅助等功能，推动平台向着高可靠、可扩展的方向持续演进，从而为住房租赁行业数字化转型提供可复用的工程实践参考。

参考文献

[1]张红菲,杨澄宇.租房货币补贴政策的福利与分配效应[J].中国经济学,2025,(04):98-123+201-203.

[2]钱丽平.基于区块链和代理重加密的网上租房系统研究[D].南京邮电大学,2021.DOI:10.27251/d.cnki.gnjdc.2021.000758.

[3]周琦.酝酿 5 年《住房租赁条例》剑指租房乱象[J].中国经济周刊,2025,(14):83-84.

[4]陈耀东.发展和规范住房租赁市场的法治保障——《住房租赁条例》要点解析[J].国家治理,2026,(02):55-59.DOI:10.16619/j.cnki.cn10-1264/d.2026.02.007.

[5]盛阳,张诗涵,孔维钾.“涓滴赋能”:人工智能驱动主流媒体国际传播的路径创新[J].国传观察,2026,(02):16-22.

[6]Baouya A ,Hamid B ,Gürgen L , et al.Rigorous Security Analysis of RabbitMQ Broker with Concurrent Stochastic Games[J].Internet of Things,2024,26101161-.DOI:10.1016/J.IOT.2024.101161.

[7]杨宇昊.我国住房租金管制法律制度研究[D].辽宁大学,2023.DOI:10.27209/d.cnki.glniu.2023.001738.

[8]黄鑫.数智时代下图书馆过刊文献利用实践研究——以郑州图书馆为例[J].图书馆学刊,2026,48(02):41-45+102.DOI:10.14037/j.cnki.tsgxk.2026.02.010.

[9]廖凤露,许毅,汤伟.图书馆数据可视化大屏系统的设计与实现[J].电脑与信息技术,2025,33(03):79-83+97.DOI:10.19414/j.cnki.1005-1228.2025.03.025.

[10]期治博.工业互联网可信数据管理与多源数据融合研究[D].北京邮电大学,2025.DOI:10.26969/d.cnki.gbydu.2025.000137.

[11]张营营.面向订单系统的规则引擎的设计和实现[D].大连理工大学,2025.DOI:10.26991/d.cnki.gdllu.2025.005415.

[12]李夏光,李晓阳.基于开源框架的企业级统一基础技术平台设计与关键技术[J].互联网周刊,2025,(23):62-64.

[13]李超,苟超.智能交通设备运维平台的研发与应用[J].通信与信息技术,2026,(02):82-86+145.

[14]Zhang W ,Li C ,Duan J .Research on the Method of Massive Data Storage Management[C]//马来西亚北方大学.Proceedings of 4th International Conference on Economic Management and Big Data Applications(ICEMBDA 2023).The People's Liberation Army 63892;,2023:511-523.DOI:10.26914/c.cnkihy.2023.123365.

[15]刘晓菲,郭保琪,刘国敬,等.工业模型互联计算方法研究[J].制造业自动化,2026,48(01):84-94.

[16]甘露平,倪晓锋,经正俊.微服务架构的电力数字化平台研究与实践[J].电脑编程技巧与维护,2026,(02):59-61+115.DOI:10.16184/j.cnki.comprg.2026.02.034.

[17]陈艳.基于 Flink 的电子书推荐系统设计与实现[J].科技创新与应用,2026,16(08):145-149.DOI:10.19981/j.CN23-1581/G3.2026.08.033.

[18]Yang L .Research on the Teaching Experiment Management System Based on JSP[C]//信息化与工程国际学会.Proceedings of 2016 6th International Conference on Machinery, Materials,Environment,Biotechnology and Computer(MMEBC 2016).Chongqing College of Electronic Engineering;,2016:363-366.

[19]滕培培,陶歆,周骅,等.热轧动态生产调度管理系统的设计与应用[J].现代信息科技,2026,10(01):169-173.DOI:10.19850/j.cnki.2096-4706.2026.01.032.

[20]刘晓伟.基于改进协同过滤和迁移学习的智能实习系统研究与开发[D].山东财经大学,2025.DOI:10.27274/d.cnki.gsdjc.2025.001411.

致谢

时光匆匆，本科生涯转眼就到了尾声。站在毕业的门槛上回望这段日子，心中有太多感触想要诉说。

首先要感谢我的指导老师。从最初选题方向的敲定，到开题报告的反复修改，再到系统开发过程中 Spring Boot 后端架构设计、协同过滤推荐算法的调优、Activiti 流程引擎集成调试等一系列技术难点的攻克，老师始终不厌其烦地给予悉心指导。每次讨论过后，我都能从困惑中理出头绪，这份严谨务实的治学态度深深影响了我。

感谢学院各位专业课老师的倾囊相授。数据库原理、软件工程、Web 开发技术等课程中学到的知识与方法，为我完成本系统的设计与实现奠定了扎实的基础。

也要感谢一路同行的同学们。实验室深夜一起联调接口、排查 Bug 的日子，食堂里边吃饭边讨论表结构设计的午后，都是这段青春岁月里最鲜活的记忆。

最后，把最深的感恩留给我的父母。正是他们在背后默默付出、无条件的理解与支持，让我能够心无旁骛地完成学业和毕业设计。带着这份感恩与力量，我将在未来的道路上继续脚踏实地、不断前行。

学生姓名：洪文豪

2026 年 6 月 1 日